cppDE: Code Generation, Integration and Sensitivity Propagation

Simon Beyer

2026-10-22

Contents

1 Overview **2**

2 Code generation **2**

3 `cppODE()` **3**
3.1 Initial-value problem . . . 3
3.2 Stiffness and the choice of method . . . 4
3.3 Multistep methods . . . 4
3.4 Single-step methods . . . 5
3.5 Adaptive step-size control . . . 7
3.6 Dense output . . . 8
3.7 Forward sensitivities . . . 8
3.8 Events and restarts . . . 10
3.9 Reverse-mode sensitivities: the discrete adjoint . . . 14
3.10 Batches of conditions . . . 22

4 `cvode()` **22**
4.1 What is shared . . . 23
4.2 What differs . . . 23
4.3 Adjoint sensitivity analysis . . . 23

5 `cppFUN()` **24**
5.1 Entry points and derivative directions . . . 24
5.2 Tangent, hessian, cotangent and curvature . . . 25
5.3 Pass-through of unmodelled inputs . . . 26

6 Benchmarks **27**

References **31**

A BDF and NDF coefficients **33**

B Adams-Moulton coefficients **33**

C Rosenbrock4 coefficients **33**

D Tsitouras 5(4) coefficients **35**

1 Overview

cppDE generates and compiles C++ code per model: for ODE models through `cppODE()` and `cvode()`, for algebraic models through `cppFUN()` (Table 1).

	`cppODE()`	`cvode()`	`cppFUN()`
problem	ODE	ODE	algebraic $y = g(x, p)$
`method`	`bdf`, `adams`, `rb4`, `tsit5`	`bdf`, `adams`	none
first order	forward, reverse	forward, reverse	forward, reverse
second order	forward-forward, forward-reverse	none	forward-forward, forward-reverse
derivatives	dual numbers, adjoint	forward sensitivity equations, adjoint	dual numbers
libraries	LAPACK, KLU optional	SUNDIALS ≥ 6.0, KLU optional	none

Table 1: The three entry points. All share the code generator of Section 2; the two integrators share `solveODE()`.

`cppODE()` and `cvode()` share their R interface: both take named character vectors of right-hand sides with optional events, forcings and parameter vectors and return a model handle for `solveODE()`. They differ only in the numerical core. `cppODE()` emits self-contained C++ that includes the headers under `inst/include/cppde/`. `cvode()` emits glue code that links the system-installed SUNDIALS CVODE/CVODES library and inherits its numerical behaviour.

Section 2 describes the shared code generator. Section 3 covers the in-tree integrator `cppODE()`: four time-stepping methods, forward-mode automatic differentiation for parameter sensitivities, the reverse-mode discrete adjoint, whose cost does not depend on the number of parameters, and sensitivities across events. Section 4 covers `cvode()` and its adjoint sensitivity analysis, and Section 5 `cppFUN()`, used for observation maps and reparametrisations $p = \Phi(\theta)$. Section 6 benchmarks the two integrators against each other on 43 problems, and the reverse and second-order modes against their forward counterparts.

2 Code generation

Every entry point turns the user's expressions into straight-line C++ templated over the scalar type, so one source serves the value, dual-number and nested-dual instantiations. The further derivatives the integrators need are formed at code-generation time by automatic differentiation [1] on an expression graph: the Jacobian, the contractions of the discrete adjoint, and the time derivatives of the right-hand side and of the event roots.

Expression graph. Each expression is read by Python's own parser and lowered into a directed acyclic graph with states, parameters, forcings, time and constants as leaves and sums, products, powers and function calls as inner nodes. Nodes are hash-consed [2], [3]: a node is identified by its operation, operands and constant attributes and is created at most once, so a repeated subexpression is a single node without a separate elimination pass. Sums and products are kept flat with numeric coefficients and exponents, and numeric constants are normalised so that equal values map to equal nodes. Syntax the parser does not read goes through SymPy into the same graph; SymPy also supplies the derivative of a function the graph has no rule for.

Example. In Figure 1 each product is one node, and so are the equal right-hand sides f_A and f_B. Nodes used more than once become temporaries:

```
const double _t0 = params[4]*x[2];       // k2*C
const double _t1 = params[3]*x[0]*x[1];  // k1*A*B
const double _t2 = _t0 - _t1;
dxdt[0] = _t2;
```

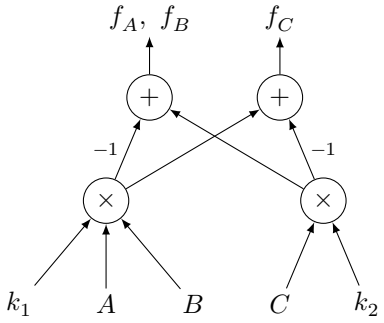


Figure 1: Expression graph of $f_A = f_B = -k_1AB + k_2C$ and $f_C = k_1AB - k_2C$; edge labels are coefficients.

```
dxdt[1] = _t2;
dxdt[2] = -_t0 + _t1;
```

Derivatives. A forward sweep over the graph forms a Jacobian-vector product, a reverse sweep a vector-Jacobian product, each in time linear in the size of the graph [1]. Both add nodes to the same graph, so derivative and value share their common parts. The reverse contractions $J^\mathsf{T}\lambda$ and $(\partial f/\partial p)^\mathsf{T}\lambda$ are vector-Jacobian products; the derivatives the Rosenbrock stages and the event localisation require are Jacobian-vector products along $\dot{x} = f$.

The Jacobian $J = \partial f/\partial x$ is assembled by entries or by colouring. By entries, each state-dependent node holds the sparse gradient of its value with respect to the states, and the gradients of the graph roots are the nonzero entries of J. The cost of this accumulation, $W_{\text{ent}} = \sum_v |\operatorname{supp} \nabla v|$ summed over the E_s state-dependent nodes v of f, grows with the density of J. Entries are used while $W_{\text{ent}} \le \max(4E_s, 50\,000)$. Beyond that, a greedy distance-2 colouring of the column intersection graph groups structurally orthogonal columns [4], [5], [6]. If it needs at most $n_x/4$ colours, one Jacobian-vector product per colour, seeded with ones on the columns of that colour, recovers those columns; otherwise entries are kept.

Linear tables. A sum of at least twelve terms $c_j\, x_j\, \varphi$ with numeric c_j and a common state-free factor φ becomes a row of a constant matrix C, so that $f(x) = \phi(x, Cx)$. The generated code evaluates $z = Cx$ once per call from a sparse or dense table, and assembles the Jacobian as

$$J = \frac{\partial \phi}{\partial x} + \frac{\partial \phi}{\partial z}\, C, \tag{1}$$

while the reverse contractions apply C^T. A dense coupling, such as a dipolar interaction among all states, thus costs a table rather than a sum of expressions per entry.

Loops. Statements whose graphs agree up to state indices and constants, such as those a method-of-lines discretisation repeats for every grid point, form a class. A class of at least sixteen instances is emitted as one loop over index and constant tables, and a sum of many terms of one structure as an inner loop. Differentiation stays per instance, so the work of the generator grows with the number of instances, while the size of the emitted code for a regular model does not.

3 `cppODE()`

3.1 Initial-value problem

`cppODE()` solves

$$\dot{x} = f(x,\, p_{\text{dyn}},\, t), \qquad x(t_0) = p_{\text{init}}, \tag{2}$$

with state $x \in \mathbb{R}^{n_x}$ and parameters $p = (p_{\text{init}}, p_{\text{dyn}})$, the initial conditions and the dynamic parameters. The right-hand side f may depend explicitly on time (non-autonomous systems), as the forcing functions that inject time-dependent inputs require. $J = \partial f/\partial x$ denotes the state Jacobian throughout. The coefficients of the four methods are listed in Appendices A to D.

3.2 Stiffness and the choice of method

Stability is analysed on the linear test equation $\dot{y} = \lambda y$, $\lambda \in \mathbb{C}^-$. A one-step method multiplies y by an amplification factor $R(z)$ per step, $z = h\lambda$. The method is *A-stable* if $|R(z)| \le 1$ on the entire left half-plane, so that the numerical solution decays whenever the exact one does, for any step size; *A(α)-stable* if the bound holds on the wedge $\{z \in \mathbb{C} : |\arg(-z)| \le \alpha\}$ for some $\alpha < 90°$; and *L-stable* if it is A-stable and $R(z) \to 0$ as $\mathrm{Re}(z) \to -\infty$, which damps very stiff transients within one step.

Problem (2) is *stiff* on $[t_0, t_{\mathrm{end}}]$ when an explicit method must take steps much smaller than local accuracy demands, to keep $h\lambda_k$ inside its stability domain, with $\{\lambda_k\}$ the eigenvalues of J along the trajectory. Stiffness thus depends on the spectrum of J, the integration horizon and the tolerance. Chemical-kinetics networks with rate constants spanning several orders of magnitude are stiff, as are many discretised parabolic PDEs.

BDF/NDF and Rosenbrock4 have stability domains containing the left half-plane or a wide wedge of it and suit stiff problems; Adams-Moulton and Tsit5 do not, at any tolerance. The cost of a wrong choice is asymmetric: an explicit method on a stiff problem fails or is forced to uneconomic step sizes, an implicit method on a non-stiff problem has a constant overhead per step. BDF/NDF is therefore the default when the spectrum of J is unknown; for a large model with a sparse Jacobian it runs on the sparse linear solver (`sparse = TRUE`).

3.3 Multistep methods

A linear multistep method determines x_n from a linear combination of the most recent $k+1$ states and right-hand-side values, $\{x_{n-j}, f_{n-j}\}_{j=0}^{k}$. Reusing past steps, it reaches high accuracy with few right-hand-side evaluations per step.

cppDE implements the BDF and Adams families in the Nordsieck representation of SUNDIALS [7]. Instead of the past $k+1$ states it stores the *scaled derivative array*

$$z_n = \left(x_n,\, h_n \dot{x}_n,\, \frac{h_n^2}{2!}\ddot{x}_n,\, \ldots,\, \frac{h_n^k}{k!} x_n^{(k)} \right).$$

A change of step size or order is then a linear transformation of z_n, not a refit through past data.

Backward Differentiation Formula (`bdf`)

The BDF of order q defines x_n implicitly by

$$\sum_{j=0}^{q} \alpha_j \, x_{n-j} \;=\; h_n \, f(x_n, t_n), \tag{3}$$

which is exact whenever $x(t)$ is a polynomial of degree at most q on the past $q+1$ nodes. It is A-stable for $q \le 2$ and A(α)-stable for $q \in \{3,4,5\}$, with α falling monotonically from 86° at $q=3$ to 51° at $q=5$.

The corrector (3) is solved by Newton iteration,

$$W \Delta x^{(\nu)} = -\tilde{r}\left(x_n^{(\nu)}\right), \qquad x_n^{(\nu+1)} = x_n^{(\nu)} + \Delta x^{(\nu)}, \tag{4}$$

with leading coefficient $\beta_0 = 1/\alpha_0$ and iteration matrix

$$W = \frac{1}{h_n \, \beta_0} \mathbb{1} - J(x_n^{(\nu)}, t_n), \tag{5}$$

in the SUNDIALS form [7], which differs from the textbook $I - h_n \beta_0 \, J$ by the scalar factor $h_n \beta_0$. Factorising W dominates the cost of a step; the factorisation is reused across steps.

cppDE defaults to the Numerical Differentiation Formula (NDF) variant of Shampine and Reichelt [8], which adds to the BDF corrector a term proportional to the Nordsieck predictor error,

$$\sum_{m=1}^{q} \frac{1}{m} \nabla^m x_n - h_n f(x_n, t_n) - \kappa_q \, \gamma_q \, (x_n - x_n^{\mathrm{pred}}) = 0, \qquad \gamma_q = \sum_{m=1}^{q} \frac{1}{m}, \tag{6}$$

with ∇ the backward-difference operator. The term scales β_0 in (4) by $(1-\kappa_q)$ and multiplies the leading truncation-error coefficient $1/(q+1)$ by $1+\kappa_q(q+1)\gamma_q$. With $\kappa_q < 0$ the truncation error at fixed step size decreases, permitting a larger step at equal accuracy. The values of κ_q (Appendix A) give the step-size gains over classical BDF reported by Shampine and Reichelt [8], 26 % for $q \le 3$ and 12 % for $q = 4$, at a small loss of stability angle. `useNDF = FALSE` recovers classical BDF ($\kappa_q \equiv 0$).

The corrector takes at most three iterations, each with one right-hand-side evaluation and one linear solve. It has converged when the damped correction norm $\delta \min(1,\rho)/\tau_q \le 1$, with δ the WRMS norm of the latest correction, ρ the estimated contraction rate and τ_q an order-dependent constant. As in CVODE, W is refactorised when $\gamma = h_n\beta_0$ has moved by more than 30 % since the last factorisation, after 20 steps, or after a failed corrector or error test; J is re-evaluated at a refactorisation once it is 51 steps old, and after a failed corrector. Without a new Jacobian, a sparse W changes only on its diagonal.

Adams-Moulton (`adams`)

The Adams-Moulton family integrates the interpolant of f through the $q+1$ samples $\{(t_{n-j}, f_{n-j})\}_{j=0}^{q}$ over $[t_{n-1}, t_n]$,

$$x_n \;=\; x_{n-1} \;+\; h_n \sum_{j=0}^{q} \beta_j \, f(x_{n-j}, t_{n-j}), \tag{7}$$

implicit in x_n through the $j = 0$ term. cppDE solves it by the Newton iteration of the BDF corrector, on $W = I - \gamma J$ with $\gamma = h/\ell_1$ from the Adams coefficients, as CVODES does with a linear solver attached: the same setup triggers, Jacobian reuse and convergence test, the same controller. Only the coefficients and the maximum order $q_{\max} = 12$ differ.

Its stability domain on the negative real axis shrinks rapidly with order; for $q \ge 3$ it contains no wedge of $\mathbb{C}^-$, so the method suits non-stiff problems with smooth solutions and a long horizon. The Newton corrector keeps a mildly stiff component from stalling the iteration, but the step size stays bounded by the stability domain.

3.4 Single-step methods

Single-step methods keep no information beyond the current state and reach order $p > 1$ through internal stages. They therefore restart after an event at full order, whereas the multistep methods rebuild the Nordsieck history from order 1.

Rosenbrock (`rb4`)

A Rosenbrock method of s stages with diagonal coefficient γ linearises the implicit Runge-Kutta condition about (x_n, t_n); its stages k_i solve

$$W k_i = f\left(x_n + \sum_{j<i} \alpha_{ij}\, k_j,\; t_n + c_i\, h_n\right) + \frac{1}{h_n} \sum_{j<i} \frac{\gamma_{ij}}{\gamma}\, k_j + h_n\, d_i\, \frac{\partial f}{\partial t}(x_n, t_n), \tag{8}$$

with iteration matrix

$$W = \frac{1}{\gamma\, h_n} \mathbb{1} - J(x_n, t_n). \tag{9}$$

Table 2 lists the coefficients.

The advanced solution and embedded error estimate are

$$x_{n+1} = x_n + \sum_{i=1}^{s} m_i\, k_i, \qquad \hat{x}_{n+1} \;=\; x_n + \sum_{i=1}^{s} \hat{m}_i\, k_i.$$

Each stage (8) is one linear solve, without Newton iteration. Per accepted step cppDE evaluates J and factorises W once, evaluates the right-hand side five times and back-substitutes six times.

cppDE uses the order-4 coefficients of Hairer and Wanner [9, Sec. IV.7] (Appendix C), the same set as the `rosenbrock4` stepper of Boost.Numeric.Odeint: an L-stable pair with an embedded order-3 estimator.

Symbol	Meaning
α_{ij}	weights with which stage i combines all earlier stages k_j, $j < i$
c_i	time abscissa of stage i inside the step, with $c_i = \sum_{j<i} \alpha_{ij}$
γ_{ij}	weights with which the linear-solve correction at stage i uses earlier stages
γ	common diagonal coefficient appearing in the iteration matrix W
d_i	weights that pick up the explicit time dependence of f via $\partial f/\partial t$
m_i	weights combining the stages into the advanced solution x_{n+1}
$\hat{m}_i$	weights combining the stages into the embedded order-3 estimator used for step control

Table 2: Rosenbrock method coefficients.

Rosenbrock methods are competitive with BDF on stiff problems whose Jacobian is cheap relative to the right-hand side. With its full-order restart, Rosenbrock4 is the stiff method of choice for systems with frequent state discontinuities at moderate state dimension.

Tsitouras (`tsit5`)

Tsitouras is the explicit Runge-Kutta pair of order 5(4) derived by Tsitouras [10] under the first column simplifying assumption alone,

$$\sum_{i=1}^{s} b_i\, a_{ij} \;=\; b_j\,(1 - c_j), \qquad j = 1, \dots, s,$$

without the row simplifying assumption $\sum_j a_{ij}\, c_j = c_i^2/2$, $i \geq 3$, on which the Dormand-Prince pair rests. Tsitouras uses the freed parameters for a smaller leading-error norm; the coefficients of Appendix D satisfy the column condition to rounding and violate the row condition. The pair has $s = 7$ stages with the First-Same-As-Last (FSAL) property,

$$k_7 \;=\; f(x_{n+1},\, t_n + h_n) \;=\; k_1 \text{ at step } n+1,$$

so an accepted step costs six right-hand-side evaluations, not seven. Table 3 defines the symbols.

Symbol	Meaning
a_{ij}	weights with which stage i combines earlier stages (strictly lower triangular)
c_i	time abscissa of stage i inside the step, with $c_i = \sum_{j<i} a_{ij}$
b_i	weights combining the stages into the advanced order-5 solution; $b_7 = 0$
$\hat{b}_i$	weights combining the stages into the embedded order-4 estimator; $\hat{b}_7 \neq 0$
k_i	right-hand side at stage i: $k_i = f\Big(x_n + h_n \sum_{j<i} a_{ij} k_j,\; t_n + c_i h_n\Big)$

Table 3: Symbols of the Tsit5 tableau.

The advanced solution and embedded error estimate are

$$x_{n+1} = x_n + h_n \sum_{i=1}^{7} b_i\, k_i, \qquad \hat{x}_{n+1} = x_n + h_n \sum_{i=1}^{7} \hat{b}_i\, k_i,$$

and the local error estimate for step-size control is

$$e_{n+1} = h_n \sum_{i=1}^{7} (\hat{b}_i - b_i)\, k_i.$$

Tsit5 needs neither Jacobian nor linear solve and, of the four methods, advances furthest per right-hand-side evaluation. It suits non-stiff problems with an expensive right-hand side.

3.5 Adaptive step-size control

The error estimate e_n of each method is reduced to a scalar by the weighted root-mean-square norm

$$\|e_n\|_{\mathrm{WRMS}} = \sqrt{\frac{1}{n_x}\sum_{j=1}^{n_x}\left(\frac{e_{n,j}}{w_{n,j}}\right)^2}, \qquad w_{n,j} = \texttt{abstol} + \texttt{reltol}\,|x^{\mathrm{ref}}_{n,j}|, \tag{10}$$

whose weights put components of different magnitude and unit on a common scale. The reference state x^{ref}_n is the accepted solution $|x_n|$ at the start of the step in BDF/NDF and Adams, as in CVODE, so the weights are fixed for the whole step, and the component-wise maximum $\max(|x_n|, |x_{n+1}|)$ in the single-step methods. A step is accepted when $\|e_n\|_{\mathrm{WRMS}} \le 1$.

The single-step methods choose the next step size with the proportional-integral (PI) controller of Hairer and Wanner [9]. With $\varepsilon_n = \|e_n\|_{\mathrm{WRMS}}$ and p the order of the embedded estimate, $p = 3$ for Rosenbrock4 and $p = 4$ for Tsit5, an accepted step sets

$$h_{n+1} = h_n\,\min\Big\{\rho_{\max},\ \max\Big(\rho_{\min},\ S\,\varepsilon_n^{-(\alpha+\beta)}\,\varepsilon_{n-1}^{\beta}\Big)\Big\}, \qquad \alpha = \frac{0.7}{p+1}, \quad \beta = \frac{0.4}{p+1},$$

with $S = 0.9$, $\rho_{\min} = 0.2$, $\rho_{\max} = 5$, and ε_{n-1} the norm of the previous accepted step, floored at 0.01. On the first step and after a rejection the integral factor $S\,\varepsilon_n^{-1/(p+1)}$ replaces the PI factor, capped at 1 after a rejection. A rejected step is retried with the factor $S\,\varepsilon_n^{-1/(p+1)}$ clamped to $[\rho_{\min}, 0.9]$.

The multistep methods choose step size and order as CVODE does. With $d_{q'}$ the WRMS norm of the error estimate at order q', the candidate ratios are

$$\eta_{q'} = \frac{1}{\left(B_{q'}\,d_{q'}\right)^{1/(q'+1)} + 10^{-6}}, \qquad q' \in \{q-1,\ q,\ q+1\},$$

with $B_{q-1} = B_q = 6$ and $B_{q+1} = 10$. The neighbouring orders are compared $q+1$ steps after a change to order q and every second step while the order is unchanged; the largest ratio fixes the next order. A ratio below 1.5 leaves h unchanged, and the ratio is capped at 10^4 on the first step and at 10 afterwards.

The order never exceeds $q_{\max}$, 5 for BDF/NDF and 12 for Adams, or a smaller `maxOrder` given to a solve. The error test alone cannot see that a step size has reached the edge of the stability domain of order q, which for $q \ge 3$ excludes a wedge around the imaginary axis: it answers with rejections and a shrinking h where a lower order would be stable. `stald = TRUE` adds CVODE's stability limit detection to the BDF family [7]. From order 3 on, each step records the squared norms of the scaled derivatives of degree $q-1$ and q, from the Nordsieck columns, and of degree $q+1$, from the corrector's correction. Once $q+5$ steps have been taken at order q, the last five records give the magnitude r of the dominant root of the step-to-step error propagation, and $r > 0.98$ lowers the order by one at the step size $\eta_{q-1}h$. It overrides the order selection unless that already lowers the order. Detection is off by default, as in CVODE: it lowers the step count on weakly damped stiff oscillations, and elsewhere it costs the estimate on every step. On a `cvode()` model, `maxOrder` and `stald` set CVODE's own `CVodeSetMaxOrd` and `CVodeSetStabLimDet`.

The first step

The single-step methods estimate the first step size from the initial state as in Hairer, Nørsett and Wanner [11], Algorithm II.4.4: a rough $h_0 = 0.01\,\|x_0\|/\|f_0\|$ fixes a scale, a second derivative $\ddot{x}$ along the trajectory fixes the curvature, and

$$h_1 = \left(\frac{0.01}{\max(\|f_0\|, \|\ddot{x}\|)}\right)^{1/2} \tag{11}$$

turns the two into a step. The exponent is that of a first-order method for both: $\|\ddot{x}\|$ stands in for the derivative of order $p+1$, which it is only at $p = 1$, and a method of higher order raises its step within a few steps. `rb4` obtains $\ddot{x} = \partial f/\partial t + Jf$ from the generated Jacobian, `tsit5` from a finite difference of f. The norms are weighted maximum norms with weights $\texttt{abstol} + \texttt{reltol}\,|x_0|$; with `sensErrCon = TRUE` they

include the sensitivity components. The step is $\min(100\,h_0,\, h_1,\, h_{\mathrm{ub}})$: the upper bound h_{ub}, at most a tenth of the interval, keeps a single Euler step from changing any component by more than a fraction of its own scale, and a lower bound keeps $t_0 + h$ distinct from t_0 in double precision.

The multistep methods use a port of the CVODES routine `cvHin`, which starts from $\sqrt{h_{\mathrm{lb}} h_{\mathrm{ub}}}$, with h_{lb} above the rounding of the time variable, and refines it by a finite-difference curvature until two successive estimates agree within a factor of two, so the first step equals the one CVODES takes on the same problem. After every event each method estimates its step again with this port, from the post-event state.

3.6 Dense output

Requested output times in general do not coincide with the adaptive step grid. Each accepted step therefore provides a local interpolant that yields $x(t)$ anywhere inside the step by polynomial arithmetic alone, without right-hand-side evaluations. With $s = (t - t_n)/h_n \in [0, 1]$ the three families use:

- Tsit5: the cubic Hermite interpolant through x_n, x_{n+1} and the FSAL derivatives k_1, k_7,

$$x(t) = (1 + 2s)(1 - s)^2\, x_n + s^2(3 - 2s)\, x_{n+1} + s(1 - s)^2\, h_n k_1 + s^2(s - 1)\, h_n k_7.$$

- Rosenbrock4: the stage-based cubic of Hairer and Wanner [9],

$$x(t) = (1 - s)\, x_n + s(x_{n+1} + (1 - s)(c_3 + s\, c_4)), \qquad c_3 = \sum_{i=1}^{5} d_i^{(2)} k_i, \quad c_4 = \sum_{i=1}^{5} d_i^{(3)} k_i,$$

  with the coefficients of Table 12.
- BDF/NDF and Adams: the Nordsieck polynomial of degree q held after the step,

$$x(t) = \sum_{j=0}^{q} z_{n+1,j} \left(\frac{t - t_{n+1}}{h_n} \right)^j.$$

Both single-step interpolants are of order 3, below the order of the step itself (5 for Tsit5, 4 for Rosenbrock4); the multistep interpolant has the order q of the step.

3.7 Forward sensitivities

Dual-number arithmetic

Parameter sensitivities of the trajectory are computed by forward-mode automatic differentiation [1], not by finite differences, whose truncation and round-off errors take several integrations to control. Every floating-point scalar is augmented by a component holding its directional derivative. Arithmetic and elementary functions are overloaded for the augmented type, so the C++ source that evaluates f is compiled unchanged for the primal and the sensitivity computation.

The augmented type is the *dual number*

$$\tilde{x} = x + x'\,\varepsilon, \qquad \varepsilon^2 = 0, \quad \varepsilon \neq 0,$$

with primal value $x \in \mathbb{R}$ and directional derivative $x' \in \mathbb{R}$. Read as a first-order Taylor expansion in ε, it obeys, for $\tilde{x} = x + x'\varepsilon$ and $\tilde{y} = y + y'\varepsilon$,

$$\tilde{x} + \tilde{y} = (x + y) + (x' + y')\varepsilon, \qquad \tilde{x} \cdot \tilde{y} = xy + (xy' + x'y)\varepsilon,$$

and for any smooth scalar function g,

$$g(\tilde{x}) = g(x) + g'(x)\, x'\, \varepsilon.$$

Compositions propagate the derivative of the composed function, so the source that evaluates f over $\mathbb{R}$ returns f and its directional derivative when called with dual inputs.

Tangent storage

A dual stores its value inline and m tangent slots behind a pointer; m is ncol of the tangent, set at run time rather than at compile time. Tangents of temporaries come from a thread-local bump arena, and everything a generated model body allocates there is released in one operation when the body returns. State vectors the stepper keeps across steps point into one contiguous $n_x \times m$ block per vector, the tangent slab, sized once per solve. A tangent wider than one call should handle is split into blocks of directions; since the step sequence of a tangent-blind solve does not depend on m, the blocks share one grid and the assembled Hessian equals that of a single pass exactly.

A dual whose width was never set has no slots and reads as a zero tangent. Buffers zeroed before use, such as the cotangents of a backward sweep, therefore get their width declared first; otherwise their derivatives would be structurally zero.

Sensitivity equations via dual evaluation

For a right-hand side depending on parameters $p \in \mathbb{R}^{n_p}$,

$$\dot{x} = f(x, p, t), \qquad x(t_0) = x_0(p),$$

the *sensitivity matrix* is

$$S_p = \frac{\partial x}{\partial p} \in \mathbb{R}^{n_x \times n_p}, \qquad (S_p)_{ij} = \frac{\partial x_i}{\partial p_j}.$$

Seeding each state component with its row of S_p gives the dual state

$$\tilde{x}_i = x_i + (S_p)_{ij}\, \varepsilon_j,$$

with dual unit ε_j for direction p_j and summation over repeated indices. The ε_j-coefficient of the i-th output of f evaluated over the dual algebra is

$$\left[f(\tilde{x}, p, t)\right]_{i,\,\varepsilon_j} = \frac{\partial f_i}{\partial x_k}\,(S_p)_{kj} + \frac{\partial f_i}{\partial p_j}.$$

The primal part recovers $\dot{x}_i = f_i(x, p, t)$; differentiating it with respect to p_j identifies the ε_j-coefficient with $(\dot{S}_p)_{ij}$, which gives

$$\dot{S}_p = \frac{\partial f}{\partial x}\, S_p + \frac{\partial f}{\partial p}, \qquad S_p(t_0) = \frac{\partial x_0}{\partial p}. \tag{12}$$

One dual evaluation of f thus yields the right-hand side of (12) for all columns of S_p, at a cost proportional to n_p and without forming $\partial f/\partial x$.

Seeding: from S_p to S_θ

Inference parameters are often a vector $\theta \in \mathbb{R}^M$ linked to the dynamic parameters p_{dyn} and the initial condition $x_0 = p_{\mathrm{init}}(\theta)$ by a reparametrisation $p = \Phi(\theta)$. The quantity of interest is then

$$S_\theta = \frac{\partial x}{\partial \theta} \in \mathbb{R}^{n_x \times M},$$

and the chain rule $S_\theta = S_p\, \partial p/\partial\theta$ turns (12) into

$$\dot{S}_\theta = \frac{\partial f}{\partial x}\, S_\theta + \frac{\partial f}{\partial p_{\mathrm{dyn}}}\, \frac{\partial p}{\partial \theta}, \qquad S_\theta(t_0) = \frac{\partial p_{\mathrm{init}}}{\partial \theta}. \tag{13}$$

The matrix $\partial p/\partial\theta$, the *tangent* of the inputs, is passed as the `tangent` argument of `solveODE()` and is fixed during integration. The choice $\partial p/\partial\theta = I_{n_p}$ gives $S_\theta = S_p$; each of the M directions of θ-space costs one additional state vector.

Error norm with sensitivities

With forward sensitivities the integrator integrates $S_\theta \in \mathbb{R}^{n_x \times M}$ beside the state, and every step produces $1 + M$ error vectors of length n_x: $e_n^{(x)}$ for the state and $e_S^{(k)}$ for each column $\partial x / \partial \theta_k$. As in CVODES, they are combined by the maximum of the per-vector WRMS norms, not by a flat WRMS over all $n_x(1 + M)$ components.

The state norm $\|e_n^{(x)}\|_{\mathrm{WRMS}}$ is (10). The column norm is its analogue with S_θ in place of x in error vector and weight,

$$\|e_S^{(k)}\|_{\mathrm{WRMS}} = \sqrt{\frac{1}{n_x} \sum_{i=1}^{n_x} \left(\frac{(e_S)_{ik}}{w_i^{(k)}} \right)^2}, \qquad w_i^{(k)} = \texttt{abstol} + \texttt{reltol}\,|(S_\theta)_{ik}^{\mathrm{ref}}|, \tag{14}$$

where $(S_\theta)_{ik}^{\mathrm{ref}}$ is taken like x_n^{ref}: the accepted value for multistep methods, the maximum over both step ends for single-step methods. The step-size controller uses the largest of these norms,

$$\|e_n\|_{\mathrm{WRMS}} = \max\left\{ \|e_n^{(x)}\|_{\mathrm{WRMS}},\ \max_{1 \le k \le M} \|e_S^{(k)}\|_{\mathrm{WRMS}} \right\}. \tag{15}$$

A flat WRMS divides by $n_x(1 + M)$, so small errors in some columns can mask a large error in another; the maximum holds each direction to the requested tolerance separately.

The BDF/NDF corrector uses the same maximum, so an unconverged sensitivity direction forces further iterations or a smaller step even when the state has converged. With `solveODE(..., sensErrCon = FALSE)`, error test, order selection, corrector test and initial-step estimate use the state components only, and the step sequence is that of a value run.

3.8 Events and restarts

Two kinds of events interrupt smooth integration. A *time-triggered event* fires at a time $t_e(\theta)$ given as a closed-form expression, which may depend on θ. A *root-triggered event* fires when an event function $g_e(x, t)$ crosses zero along the trajectory, at the time $t^*(\theta)$ defined implicitly by $g_e(x(t^*), t^*) = 0$. Either kind applies a discontinuous reset

$$x^+ = g(x^-, t),$$

such as dosing a species, clipping to non-negative values, or switching between dynamical regimes.

Restart cost

Single-step methods (Rosenbrock4, Tsit5) resume after the reset with a full-order step. Multistep methods (BDF/NDF, Adams) restart at order 1 and rebuild the Nordsieck array over several steps, which dominates the cost on problems with frequent events. For every method the step size is re-estimated from the post-event state by the initial-step estimator of the solve, bounded by the remaining interval; for a single-step method this is the whole restart.

Sensitivity transport across a discontinuity

The reset g acts on the trajectory $x(t, \theta)$ at the firing time $t^*(\theta)$. The linearised update $S_\theta^+ = (\partial g / \partial x)\, S_\theta^-$ misses the θ-dependence of the firing time: a perturbation δt^* adds short integrations of $\dot{x} = f$ on either side of the event, with different right-hand sides before and after the reset. With f^- the right-hand side at x^- and f^+ that at $g(x^-, t^*)$, the sensitivity at the first grid time after the event receives two contributions beyond the linearised reset:

1. *Pre-event drift.* x^- flows along $\dot{x} = f^-$ from the grid time $t_*^{(0)} := \mathrm{scalar}(t^*)$ to the perturbed firing time t^*, over $\delta t^* = t^* - t_*^{(0)}$, whose scalar part vanishes but whose θ-derivatives do not.
2. *Post-event drift.* The reset state $g(x^-, t^*)$ flows along $\dot{x} = f^+$ from t^* back to the grid time, over $-\delta t^*$.

Both contribute terms of order $f^{\pm}\,(\partial t^*/\partial\theta)$; to first order they give the saltation matrix of Section 3.8.

cppDE replaces the bare reset by forward drift, reset and backward drift, evaluated in dual arithmetic. Each drift is one step of Heun's method, the explicit trapezoidal rule. Since δt^* has scalar part zero, the value of x at the grid time is unchanged and only the θ-derivatives of the displacement enter the state. The reset between the drifts is evaluated in the same arithmetic, so its first and second derivatives enter as well. Without sensitivities, g is applied as a plain map.

The two kinds of event differ in how δt is obtained: from the expression $t_e(\theta)$, or by the implicit function theorem applied to $g_e = 0$.

Time-triggered events

For a firing time given as an expression $t_e(\theta)$, the θ-derivatives follow from the chain rule. The integrator runs to the scalar grid time $t_e^{(0)} := \mathrm{scalar}(t_e)$ and treats the displacement

$$\delta t = t_e - t_e^{(0)}, \qquad \mathrm{scalar}(\delta t) = 0, \tag{16}$$

as a dual number of value zero with first-order tangents $\partial t_e/\partial\theta_a$ and second-order tangents $\partial^2 t_e/\partial\theta_a\partial\theta_b$. The state at $t_e^{(0)}$ is x^-; its first-order tangents are the pre-event sensitivities S_θ. The transport, a Heun step forward to t_e, the reset, and a Heun step back to $t_e^{(0)}$ with the post-event right-hand side, reads

$$\begin{aligned}
&\text{forward:} \quad f_1 = f(x^-, t_e^{(0)}), \quad x_E = x^- + f_1\,\delta t, \quad f_2 = f(x_E, t_e),\\
&\qquad\qquad x^* = x^- + \tfrac{1}{2}(f_1 + f_2)\,\delta t,\\
&\text{reset:} \quad x_+^* = g(x^*, t_e),\\
&\text{backward:} \quad \tilde f_1 = f(x_+^*, t_e), \quad x_B = x_+^* - \tilde f_1\,\delta t, \quad \tilde f_2 = f(x_B, t_e^{(0)}),\\
&\qquad\qquad x^+ = x_+^* - \tfrac{1}{2}(\tilde f_1 + \tilde f_2)\,\delta t.
\end{aligned} \tag{17}$$

An Euler step would suffice at first order: since $\mathrm{scalar}(\delta t) = 0$, the powers δt^k, $k \geq 2$, have no first-order dual content, so every $O(\delta t^2)$ term drops out of $\partial x^+/\partial\theta$. At second order the curvature term

$$\tfrac{1}{2}(J\,f + \partial f/\partial t)\,\delta t^2, \qquad J = \partial f/\partial x,$$

keeps a second-order dual component, because $(\delta t)^2$, though zero in value and first derivatives, has the mixed derivative

$$\frac{\partial^2(\delta t)^2}{\partial\theta_a\,\partial\theta_b} = 2\,\frac{\partial t_e}{\partial\theta_a}\,\frac{\partial t_e}{\partial\theta_b}.$$

Heun's second evaluation, at the shifted state and the shifted time t_e, supplies this term and leaves a truncation error of $O(\delta t^3)$. Evaluating f_2 and $\tilde f_2$ at $t_e^{(0)}$ instead would drop the $\frac{1}{2}(\partial f/\partial t)\,\delta t^2$ part for a right-hand side that depends explicitly on time.

Root-triggered events

A root event fires at the time $t^*(\theta)$ defined implicitly by

$$G(t^*(\theta), \theta) := g_e(x(t^*(\theta), \theta), t^*(\theta)) = 0. \tag{18}$$

It is handled in three steps: localisation of t^* on the scalar trajectory, reconstruction of the dual residual δt^* by the implicit function theorem, and the Heun transport (17).

Localisation The integrator evaluates g_e at the end of every accepted step and at every requested output time. A crossing on $[t_n, t_{n+1}]$ is accepted when the two readings differ in sign, or when $g_e(x_{n+1}, t_{n+1})$ is exactly zero, as for a root on a sampled time. A zero at the lower end was reported with that sample and is not a new crossing; this also keeps an event from firing again on the surface it was just applied on, where g_e is zero up to round-off.

The crossing is refined on the dense output of the accepted step, without further right-hand-side evaluations, by false position with the interpolation factor clamped to $[0.1, 0.9]$. The clamp bounds the stagnation of the classical rule and keeps its speed on a nearly linear g_e. The iteration stops when the bracket is narrower than the root tolerance, after at most fifty iterations, or when an iterate gives $g_e = 0$ exactly. It returns as $t_*^{(0)} := \mathrm{scalar}(t^*)$ the end of the final bracket that still has the pre-crossing sign of g_e. Localisation yields no θ-dependence of t^*, so the dual residual $\delta t^* = t^* - t_*^{(0)}$ is reconstructed from (18).

Dual residual of the event time If G is smooth at the crossing and $\partial G/\partial t \neq 0$ there, the implicit function theorem applied to (18) gives

$$\frac{\partial t^*}{\partial \theta} = -\frac{G_\theta}{G_t} = -\left.\frac{\nabla_x g_e \cdot S_\theta}{\nabla_x g_e \cdot f + \partial g_e/\partial t}\right|_*, \tag{19}$$

whose denominator is the total time derivative $\dot{g}_e = \nabla_x g_e \cdot f + \partial g_e/\partial t$ along the trajectory at the crossing. The condition $\dot{g}_e \neq 0$ means a transversal crossing of the event surface; at a grazing crossing the firing time is not differentiable in θ, and the fallback under simultaneous events applies. In dual arithmetic (19) is the quotient

$$\delta t^* = -\frac{g_e(x^-, t_*^{(0)})}{\dot{g}_e(x^-, t_*^{(0)})}, \tag{20}$$

with g_e and $\dot{g}_e$ evaluated on the dual state x^- at the scalar grid time. The generated partials $\nabla_x g_e$ and $\partial g_e/\partial t$ are needed only for $\dot{g}_e$; the dual arithmetic forms $\nabla_x g_e \cdot S_\theta$ in the numerator. The scalar value of the numerator, of the order of the root tolerance, is set to zero before the division, so δt^* is zero in value and has only derivatives.

The first-order part of (20) equals (19). At second order the quotient lacks the G_{tt} contribution, because g_e is evaluated at the scalar time $t_*^{(0)}$, not at the dual time t^*. Expanding (18) to second order in θ with $\delta t^* = \delta t^*_{(1)} + \delta t^*_{(2)} + \cdots$ and $G(t_*^{(0)}, \theta^{(0)}) = 0$ gives

$$0 = G_t\, \delta t^*_{(2)} + \tfrac{1}{2} G_{tt}\, (\delta t^*_{(1)})^2 + G_{t\theta}\, \delta t^*_{(1)}\, \delta\theta + \tfrac{1}{2} G_{\theta\theta}\, \delta\theta^2,$$

of which the quotient supplies the $G_{t\theta}$ and $G_{\theta\theta}$ terms. The corrected update is

$$\delta t^* \leftarrow \delta t^* - \tfrac{1}{2}\, \frac{G_{tt}}{G_t}\, (\delta t^*)^2, \tag{21}$$

where

$$G_{tt} = \left.\frac{\mathrm{d}^2}{\mathrm{d}t^2} g_e(x(t), t)\right|_*$$

is the second total time derivative of g_e along the trajectory, taken from generated code or, where that is not available, from a forward finite difference of $\dot{g}_e$. The correction (21) changes only the second-order components of δt^*.

The Heun transport (17) then moves x^- across the event to the grid time $t_*^{(0)}$, with δt^* in place of δt and $t_*^{(0)} + \delta t^*$ in place of t_e.

Simultaneous events Several root conditions can cross zero within one accepted step. Each crossing is localised separately, and those that share $t_*^{(0)}$ within the localisation tolerance form a cluster. For a cluster, one forward Heun shift reaches the common event surface, every triggered reset is applied there (the reset maps are assumed to commute), and one backward shift returns to the grid time: four right-hand-side evaluations regardless of the number of events. The dual residual δt^* is computed once, from the first event in the cluster with non-zero $\dot{g}_e$, and shared by the others, which sit at the same grid time with the same parameter dependence through x^-.

If every event in the cluster has $\dot{g}_e = 0$ at the surface, the denominator of (19) vanishes and the implicit function theorem gives no $\partial t^*/\partial\theta$. cppDE detects this case by the size of $|\dot{g}_e|$ and applies the reset as a plain map without transport, which is exact in the limit where both contributions $f^{\pm}\,(\partial t^*/\partial\theta)$ vanish.

Conditions switched by a reset A root condition that a reset makes true escapes the sign-change search, since a jump is not a crossing of the continuous solution. Every condition is therefore evaluated on both sides of a jump. A condition whose sign changes across the jump fires there, one per sweep so that a reset can switch on the next condition; one that already held before the jump does not fire again.

The switched resets are applied at the jump, between the forward and the backward shift of the event that caused it, not after the transport has returned to the grid time. One Heun transport then applies the composition of both resets, and its first- and second-order sensitivities equal those of the same reset written as a time-triggered event at that time.

Terminal conditions are excluded: they stop the integration instead of resetting the state.

Switches on states and time

A comparison in the right-hand side whose sides differ by an amount $g(x,t)$ that reads a state or the time, and Heaviside(g) or sign(g), switch f on the set where g changes sign. Read as written, such a condition makes f discontinuous in x: a step across the switch fails the error test until it is short enough, and the sensitivities miss the dependence of the switching time on θ altogether.

The code generator therefore holds each such condition as a *mode* $m \in \{0,1\}$, which holds where $g > 0$ (or $g \geq 0$ for a non-strict comparison); $a < b$ becomes the mode of $g = b - a$. The right-hand side, its Jacobian and every derivative code read the mode in place of the comparison, so between two switches $f(x,t;m)$ is as smooth as its branches. Heaviside(g) and sign(g) take two modes, on g and on $-g$, so that they keep their value at $g = 0$. Equality and inequality are evaluated as written, since they switch on a set of measure zero.

A switch is then a root event with g as its condition and no reset. The step that crosses $g = 0$ integrates the branch it started on, which is smooth there, and the crossing is localised on the dense output of that step as in the root-event case, to a few doubles in t rather than to the root tolerance, since the switching time enters the state through the jump of f. The mode is set on the event surface, between the forward and the backward Heun shift, so the transport (17) reads $f^- = f(\,\cdot\,; m^-)$ on the near side and $f^+ = f(\,\cdot\,; m^+)$ on the far side, and the reset is the identity. To first order this is the saltation matrix of Section 3.8 with $\partial g/\partial x = I$,

$$S_\theta^+ = S_\theta^- + (f^- - f^+)\,\frac{\partial t^*}{\partial \theta}, \qquad \frac{\partial t^*}{\partial \theta} = -\frac{\nabla_x g \cdot S_\theta^- + \partial g/\partial \theta}{\nabla_x g \cdot f^- + \partial g/\partial t},$$

and the second-order components follow from the same dual arithmetic, with the G_{tt} correction (21) taken along f^-. Where f is continuous across the switch, a kink, the first-order jump vanishes and the transport still supplies the second-order term, which comes from the jump of $\partial f/\partial x$ over the shifts.

The modes are read off the state at the start of the solve and after every reset, since a jump can move the state across a switch; a switch crossed by its root takes the side its root crossed to. A switch adds no output row and is not limited by `maxroot`. A root that is zero at the start of a step sits on its surface and shows no sign change when the state leaves it to the side its mode does not describe; such a departure is taken as a crossing once $|g|$ exceeds its residual and rounding there. When that happens twice in a row the solution slides along the surface, both branches pointing at it, and the solve stops with an error: a sliding motion is not a solution of either branch.

The reverse mode keeps the modes each step ran under with its checkpoint, sets them before the step's adjoint, and transposes the switch's sandwich as that of a root event (Section 3.9), with f^- and its derivatives read in the old mode and f^+ in the new one. `cvode()` hands g to CVODES' root finding, sets the mode at the root it returns, adds $(f^- - f^+)\,\partial t^*/\partial\theta$ to the sensitivities before `CVodeSensReInit()`, and under adjoint sensitivity analysis starts a new stretch there, whose backward problem reads the modes the stretch ran under.

Connection to the saltation matrix

The classical *saltation matrix* adds to the reset Jacobian $\partial g/\partial x$ a term for the θ-dependence of the firing time:

$$S_\theta^+ = \frac{\partial g}{\partial x}\,S_\theta^- + \left(\frac{\partial g}{\partial t} + \frac{\partial g}{\partial x}\,f^- - f^+\right)\frac{\partial t^*}{\partial \theta}.$$

The first-order part of the Heun transport equals this expression, with $\partial t^*/\partial\theta = \partial t_e/\partial\theta$ for time-triggered events and $\partial t^*/\partial\theta$ from (19) for root events. The transport also yields the second-order sensitivities, which the saltation matrix does not. It is implemented in `cppde_saltation.hpp`, the event loop in `cppde_event_engine.hpp`.

3.9 Reverse-mode sensitivities: the discrete adjoint

Forward sensitivities cost one additional state vector per direction in θ, so the gradient of a scalar objective over M parameters costs M times the value alone. For a single scalar J, such as a likelihood or a residual sum of squares, the transposed computation delivers the whole gradient at a cost independent of M [1], [12]. cppDE builds this mode with `cppODE(..., derivMode = "reverse")`.

The reverse mode computes a contraction, not the sensitivity matrix. Given a cotangent $W \in \mathbb{R}^{n_t \times n_x}$ on the observed trajectory, one sweep returns

$$\bar{\theta}_j = \sum_{o=1}^{n_t} \sum_{i=1}^{n_x} W_{oi} \frac{\partial x_i(t_o)}{\partial \theta_j}, \qquad j = 1, \dots, n_x + n_p, \tag{22}$$

where o runs over the observation times and i over the state components. Row o of W is written $w_o \in \mathbb{R}^{n_x}$, the cotangent at t_o. For a least-squares objective W is the residual weighted by the error model, and $\bar{\theta}$ is $\nabla_\theta J$ exactly. Several cotangents may be supplied as columns of a three-dimensional array, one sweep each.

The adjoint equation

The contraction (22) can be evaluated without forming S_p, the forward sensitivity of (12). Both backends implement the classical construction below, with different discretisations. Define $\lambda(t) \in \mathbb{R}^{n_x}$ backwards from the end of the observed interval by

$$\dot{\lambda} = -J(x,p,t)^{\mathsf{T}}\lambda, \qquad \lambda(T^+) = 0, \qquad \lambda(t_o^-) = \lambda(t_o^+) + w_o, \tag{23}$$

so that λ is continuous between observation times and jumps by the cotangent row at each. Along the solution the two Jacobian terms cancel,

$$\frac{\mathrm{d}}{\mathrm{d}t}\left(\lambda^{\mathsf{T}} S_p\right) = -\lambda^{\mathsf{T}} J S_p + \lambda^{\mathsf{T}}\left(J S_p + \frac{\partial f}{\partial p}\right) = \lambda^{\mathsf{T}} \frac{\partial f}{\partial p}, \tag{24}$$

so λ is defined without reference to θ. Integrating (24) over $[t_0, T]$ and collecting the jumps, which contribute exactly the terms of (22), gives

$$\bar{\theta} = \left(\frac{\partial x_0}{\partial \theta}\right)^{\mathsf{T}} \lambda(t_0) + \left(\frac{\partial p}{\partial \theta}\right)^{\mathsf{T}} \int_{t_0}^{T} \left(\frac{\partial f}{\partial p}\right)^{\mathsf{T}} \lambda(t)\,\mathrm{d}t. \tag{25}$$

When θ consists of the initial state and the parameters themselves, both outer factors are identities and a reverse solve reports the pair

$$\bar{x}_0 = \lambda(t_0), \qquad \bar{p} = \int_{t_0}^{T} \left(\frac{\partial f}{\partial p}\right)^{\mathsf{T}} \lambda(t)\,\mathrm{d}t. \tag{26}$$

A reparametrisation $p = \Phi(\theta)$ is applied afterwards through the outer factors of (25), the chain rule of (13) applied on the left.

The gradient costs one backward solve of a linear system of dimension n_x and a quadrature of dimension n_p, whatever M; $\partial f/\partial p$ enters only contracted with λ and is never formed. The two parts of (26) are returned stacked in the row order of the `tangent` argument, states first, then parameters.

Discrete and continuous adjoints

Equation (23) describes the exact trajectory; cppDE provides both common discretisations of it. The *continuous* adjoint differentiates, then discretises: (23) is solved backwards by an integrator of its own over forward states recovered by interpolation [13]. This is the route of CVODES, exposed by `cvode()` (Section 4.3). The *discrete* adjoint, implemented by `cppODE()`, reverses that order: it transposes the map the forward run applied, one accepted step at a time. Both approximate the same gradient and converge as the tolerance is lowered. They differ in three respects.

Consistency of value and gradient. The discrete adjoint differentiates the computed trajectory and returns the derivative of the reported value, up to rounding. The continuous adjoint approximates the derivative of the exact solution, which no run computes, so its gradient is consistent with the accompanying value only to $O(\mathrm{tol})$, and tightening the backward tolerance alone does not remove the difference.

Discontinuities. A jump in the state implies a jump in λ, given by the transposed saltation matrix of the event. The continuous adjoint restarts its backward integration with that matrix at every event; in the discrete adjoint the jump is the transpose of arithmetic the forward run performed, handled like any other step.

Linear algebra per step. The discrete adjoint of an implicit step is one transposed solve against the step's iteration matrix, evaluated at the checkpointed state, with neither Newton iteration nor error control. The continuous adjoint integrates a stiff linear ODE with its own step-size control and step count.

The step map and its adjoint

Reverse accumulation has no counterpart in an augmented arithmetic: a dual number holds its derivative with its value, available as soon as an operation has executed, whereas an adjoint sums over all later uses of a quantity. A step's adjoint is obtained either by recording its operations on a tape and traversing it backwards, or in closed form, since the map is one accepted step of a known method on a fixed grid. cppDE uses the closed form and keeps no tape.

An accepted step of the multistep family decomposes into the Nordsieck rescale and Pascal shift, the corrector, and the tail the controller applies between the acceptance of this step and the next:

$$z^{\mathrm{pred}} = A\, z^{\mathrm{in}}, \tag{27}$$

$$r(y, z^{\mathrm{pred}}, \theta) = 0, \tag{28}$$

$$z^{\mathrm{out}} = B\, z^{\mathrm{pred}} + c\, a, \qquad a = y - z_0^{\mathrm{pred}}. \tag{29}$$

The operators A, B and c act on the slot index only; their coefficients follow from the Nordsieck carry, the order, step-size history and counters the stepper holds beside the slots, and are the same for every state component. They are read off the stepper, not transposed by hand: a probe whose states are the Nordsieck slots is driven through the routines of the forward step, seeded with the identity in the [slot $\times$ component] array, and one pass returns every column. The adjoint therefore uses exactly the coefficients of the forward step.

The corrector is differentiated by the implicit function theorem. Since $\partial r/\partial y$ is the iteration matrix $W = I - \gamma J$ the forward step factorises, one transposed solve against W replaces the Newton iteration. For the Rosenbrock family the counterpart is a transposed stage recursion, six solves against one factorisation; for an explicit method, the backward recursion over the tableau.

The code generator emits the contractions the adjoint requires beside the right-hand side: $J^{\mathsf{T}}\lambda$, $(\partial f/\partial p)^{\mathsf{T}}\lambda$ and, for a Rosenbrock method, the same two applied to Jv, which contain a second derivative because the stages solve against a matrix built from the Jacobian.

Checkpoint per step

The forward run stores one checkpoint per accepted step: the step-start state, the carry, the time, the step size, and a record of the operations the controller applied to the carry afterwards. Memory is linear in the

number of steps and holds no derivatives. The factorisations of the forward run are not kept, so the sweep re-forms the iteration matrix W_k at each checkpoint. The slot operators are cached, keyed on the carry and the step size, so the probe runs only where the controller changed the control state.

The transposed solve $W_k^{\mathsf{T}}\mu = b$ goes through the factorisation of a later step's matrix W_h, which the sweep holds, by iterative refinement,

$$\mu \leftarrow \mu + W_h^{-\mathsf{T}}(b - W_k^{\mathsf{T}}\mu), \qquad W_k^{\mathsf{T}}\mu = \tfrac{1}{\gamma}\,\mu - J_k^{\mathsf{T}}\mu,$$

until the correction is at rounding level, so the answer is that of a direct solve. Each pass costs a pair of triangular solves and a product with J_k^{T}. A pass is cheaper than a factorisation by the ratio of their floating-point work, which KLU reports, and the sweep refines only while that ratio allows at least three passes; where the iteration converges too slowly or exhausts that budget, W_k is factorised and becomes the held matrix. The budget depends on the factors alone, so a sweep is reproducible.

Observations lie at interpolated times and are evaluated through the continuous extension of the step containing them. The adjoint of that evaluation comes from the same probe: after the tail, the probe holds a valid interpolant, and evaluating it at the observation time yields the row $\partial x_{\text{interp}}/\partial(z^{\text{pred}}, a)$ directly.

Checkpoint reuse

The sweep reads only the checkpoints, so the forward and backward passes can be separate calls. A reverse solve with `keepStore = TRUE` returns its checkpoints as `$store`; a later solve of the same model at the same times and parameters accepts them through `store`, performs no integration and proceeds directly to the sweep. A store created at a different parameter value is rejected. The call that creates a store may omit the cotangent and then performs a value-only run.

A gradient needs two solves at the same parameter value, one for the values from which the cotangent is formed and one for the sweep; the store removes the second integration. The columns of a three-dimensional cotangent sweep the one store in turn, so n_{seed} cotangents cost n_{seed} sweeps and a single integration.

Under `derivMode = "forward-reverse"` both arguments are refused: a checkpoint of that mode holds tangents, which live in the arena of the recording solve and do not outlive it, so every second-order sweep is preceded by its own integration.

The adjoint of an event

An event is a map of its own between two steps. The forward run transports the discontinuity with the Heun sandwich of Section 3.8: a shift to the event surface, the resets, and a shift back to the grid time. Each shift is taken over the residual of the event time, which is zero in value and has only derivatives. In value the jump is therefore a plain jump, and the sandwich reduces to the classical saltation relation

$$\mathrm{d}x = R'(\mathrm{d}x_{\text{before}} + f_{\text{before}}\, s) - f_{\text{after}}\, s, \tag{30}$$

with R the resets, R' their Jacobian and s the differential of the event time. For a root event $g(x,t,p) = 0$ the implicit function theorem gives $s = -(\nabla g \cdot \mathrm{d}x + \partial g/\partial p \cdot \mathrm{d}p)/\dot{g}$; for a fixed event at $t = \tau(p)$, $s = (\partial\tau/\partial p)\cdot \mathrm{d}p$.

Equation (30) is linear in $\mathrm{d}x_{\text{before}}$ and $\mathrm{d}p$, so its transpose can be read off. Contracting it with a cotangent λ_{after} and collecting the terms that multiply s leaves one scalar κ,

$$\kappa = \lambda_{\text{after}}^{\mathsf{T}}\,(R'\,f_{\text{before}} - f_{\text{after}})\,, \qquad \lambda_{\text{before}} = R'^{\mathsf{T}}\lambda_{\text{after}} + \kappa\left(\frac{\partial s}{\partial x}\right)^{\mathsf{T}}, \tag{31}$$

and the parameter cotangent additionally receives $\kappa\,(\partial s/\partial p)^{\mathsf{T}}$. For a root event both derivatives of s contain the factor $-\kappa/\dot{g}$: the state picks up $-(\kappa/\dot{g})\nabla g$ and the parameters $-(\kappa/\dot{g})\,\partial g/\partial p$. For a fixed event $\partial s/\partial x = 0$ and the parameters pick up $\kappa\,\partial\tau/\partial p$.

The transpose R'^{T} is applied reset by reset, in reverse order. A reset that replaces a state removes the incoming cotangent of that component, one that adds to it leaves the cotangent unchanged, and one that

multiplies it by h scales it by h; where h depends on the state or the parameters, its derivatives enter the same transpose. The adjoint of a jump therefore requires the right-hand side at both ends, ∇g and $\dot{g}$, and the model's derivatives of the reset and event-time expressions, but no Jacobian of f: every term the two shifts would contribute has the residual as a factor, and that factor is zero.

After a jump the multistepper restarts: `initialize` builds the Nordsieck history from a single state as $z_0 = x$ and $z_1 = h\,f(x,t)$, with the remaining slots zero, so the cotangent of the history collapses onto that state. With w_j the cotangent of slot j, the restart contributes

$$\bar{x} = w_0 + h\,J^{\mathsf{T}} w_1, \qquad \bar{p} = h\left(\frac{\partial f}{\partial p}\right)^{\mathsf{T}} w_1, \tag{32}$$

and slots above the first contribute nothing. The same map applies at the start of the trajectory.

Second order of an event's adjoint

The differential s of the event time is zero in value, with $\mathrm{d}s \neq 0$, so

$$s^2 = 0, \qquad \mathrm{d}(s^2) = 2s\,\mathrm{d}s = 0, \qquad \mathrm{d}^2(s^2) = 2\,(\mathrm{d}s)^2 \neq 0, \tag{33}$$

and terms of order s^2 drop from the gradient but not from the Hessian. Three such terms enter a jump. The same three enter a fixed event, with the event time's own residual $\tau = \varepsilon(\theta) - \lfloor \varepsilon \rceil$ in place of s: a fixed event is the same five-stage sandwich (39), and its transpose is that program reversed with τ substituted throughout. The classical saltation alone is exact in value but wrong in the tangents once the jump height moves with a parameter, because $\bar{x}_a = (I - \tau J_a)^{\mathsf{T}} \bar{x}_o$ then has a second factor that is not zero in value.

The first term is the curvature of the shift. Each half of the sandwich is a Heun step,

$$x_* = x_{\text{before}} + f_{\text{before}}\,s + \tfrac{1}{2}\,a_{\text{before}}\,s^2, \qquad a = \frac{\partial f}{\partial x} f + \frac{\partial f}{\partial t}, \tag{34}$$

and $a\,s$ is the difference of the two right-hand sides the step already evaluates, so no Jacobian is needed.

The second is the post-jump state, which the sweep reconstructs because the store keeps only x_{before}. Rebuilt from the bare reset it would satisfy $\partial x_{\text{after}}/\partial t^* = 0$, whereas

$$\frac{\partial x_{\text{after}}}{\partial t^*} = f_{\text{before}} - f_{\text{after}}, \tag{35}$$

so the reconstruction runs the sandwich, not the reset.

The third comes from the quadrature. The parameter cotangent

$$\bar{p}_k = \int_0^T G_k\,\mathrm{d}t = \int_0^{t^*} + \int_{t^*}^T, \qquad G_k = \left(\frac{\partial f}{\partial p_k}\right)^{\mathsf{T}} \lambda, \tag{36}$$

is split at t^*, where G_k jumps with the state. Differentiating a split integral with a moving split point leaves the jump of the integrand at the split,

$$\frac{\partial \bar{p}_k}{\partial v} \;\supset\; \Big(G_k(t^{*-}) - G_k(t^{*+})\Big)\,\frac{\partial t^*}{\partial v}. \tag{37}$$

Only parameters whose $\partial f/\partial p_k$ depends on a state the reset changed contribute.

Implemented as accumulations scaled by the residual, all three are zero in value and hold $\partial t^*/\partial v$ in their tangent, so the gradient is unchanged. For a fixed event at $t = \tau(p)$ the residual is $\tau - \lfloor \tau \rceil$; for a root event it is

$$s = -\,\frac{g - \lfloor g \rceil}{\dot{g}}, \qquad \dot{g} = \frac{\partial g}{\partial t} + \nabla g \cdot f, \tag{38}$$

read at the stored state. Localisation ignores the tangents, so that state holds $\partial x/\partial\theta$ at a fixed time and ∇g contracts it to the event time's.

Transposing the root sandwich

The jump is the composition

$$x_e = x_b + f_b s, \quad x_* = x_b + \tfrac{1}{2}(f_b + f_e)s, \quad x_a = R(x_*), \quad x_k = x_a - f_a s, \quad x_o = x_a - \tfrac{1}{2}(f_a + f_k)s, \tag{39}$$

with $f_e = f(x_e, t^*)$, $f_a = f(x_a, t^*)$ and $f_k = f(x_k, t_0)$. This straight-line program is transposed assignment by assignment, not through a collected closed form: a collected form pruned by order in s, keeping a product only where one factor does not vanish in value, is valid for the gradient but, by (33), not for the Hessian.

Reversed this way, the shift receives a contribution from every assignment in which it occurs, including two from evaluating f_e and f_a at t^*, which depends on θ:

$$\bar{s} \;\supset\; \left(\frac{\partial f}{\partial t}\right)_{(x_a, t^*)} \cdot \bar{f}_a + \left(\frac{\partial f}{\partial t}\right)_{(x_e, t^*)} \cdot \bar{f}_e. \tag{40}$$

Both cotangents contain a factor s, so (40) is absent from the gradient and enters the Hessian at full strength. It vanishes identically for an autonomous right-hand side; a pruned form omits it whenever $\partial f/\partial t \neq 0$.

The forward path corrects the shift by $-\frac{1}{2}(\ddot{g}/\dot{g})\, s^2$, whose chain factor $1 + 2c_2 s$, $c_2 = -\ddot{g}/(2\dot{g})$, is one in value and a second-order tangent; the transpose applies it to $\partial s/\partial x$.

The sweep binds the tangent storage of every buffer it hands to a generated contraction (Section 3.9). In the jump transpose every read of such a buffer is multiplied by s, so an unbound buffer would leave value and gradient intact and make the Hessian wrong by a rank-one term.

Derivative of the shift. Since $\dot{g}$ depends on the state,

$$\frac{\partial s}{\partial x} = -\frac{\nabla g}{\dot{g}} - \frac{s}{\dot{g}} \nabla \dot{g}, \qquad \nabla \dot{g} = \nabla \frac{\partial g}{\partial t} + (\nabla^2 g)\, f + J^{\mathsf{T}} \nabla g, \tag{41}$$

and the second term contains g, which is zero in value: absent from the gradient, full strength in the Hessian. The generator differentiates $\dot{g} = \partial g/\partial t + \nabla g \cdot f$ as one expression rather than assembling three terms, so all three are present, including the two that vanish for a g linear in the state and independent of time. This yields $\nabla \partial g/\partial t$ and $(\nabla^2 g) f$ and, on θ, the matching $\partial^2 g/\partial\theta\, \partial t$, $(\partial \nabla g/\partial\theta) \cdot f$ and $(\partial f/\partial\theta)^{\mathsf{T}} \nabla g$. A forcing inside g is differentiated in t through its rate: the time derivative on the expression graph seeds each forcing leaf with a rate leaf, which prints as the derivative of the PCHIP interpolant.

Time-dependent resets. A reset value h that depends explicitly on time is applied on the surface, at t^*, so

$$\frac{\partial x_a}{\partial t^*} \;\supset\; c\, \frac{\partial h}{\partial t}, \qquad c = \begin{cases} 1 & \text{add} \\ 1 & \text{replace} \\ x_{*k} & \text{multiply,} \end{cases} \tag{42}$$

which contributes to κ of (31). This term is first order: omitting it makes the gradient wrong, not only the Hessian. For the same reason the transposed reset receives the event time as a dual rather than a plain number, so that $\partial h/\partial\theta$ includes the dependence of t^* on θ. Such a value evaluates t^* directly and inherits the root tolerance rather than the integration tolerance; in a value that depends only on the state at t^* the localisation error is damped by the smoothness of the trajectory.

Verification. With $R = \mathrm{id}$ the composition (39) is the identity and the transpose cancels term by term. Against a nested-dual evaluation of the same sandwich, without solver, store or event engine, the transpose is exact to $9 \cdot 10^{-16}$. Over a whole trajectory, compared with forward over forward on one recorded step sequence, the second order agrees to the tolerance of the solve. Only this last check is sensitive to (40): the nested-dual check compares against the same forward routine the transpose reverses, so a term absent from both sides cancels.

Verifying a derivative across a moving event

When t^* depends on a parameter, the reported trajectory contains a row whose time moves with it. A functional $w^\mathsf{T}x$ seeded on that row differs between parameter values and has no derivative to compare against. Restricting the cotangent to the user's own output times restores a well-posed functional, and the forward, forward-over-forward and difference-quotient answers then agree to the tolerance of the solve.

Internal numerical differentiation

The step sizes the controller selected are constants of the differentiated map. This is Bock's internal numerical differentiation [14], [15]: the nominal run adapts freely, and the derivative is taken of the scheme that run applied. Differentiating the control law instead adds derivatives of the step sizes to the chain rule and makes the discrete adjoint inconsistent with the adjoint ODE [16], [17]; that term, measured at $O(\mathrm{tol})$, is left out.

The usual justification, that h is piecewise constant in θ with zero derivative almost everywhere, does not hold for an adaptive controller, and the argument below does not rely on it. Let $y_h(\theta)$ be the result the program returns on the grid the controller selects at θ. Then

$$\frac{\mathrm{d}y_h}{\mathrm{d}\theta} = \frac{\partial y_h}{\partial \theta} + \sum_k \frac{\partial y_h}{\partial h_k}\,\frac{\mathrm{d}h_k}{\mathrm{d}\theta}, \tag{43}$$

where internal numerical differentiation computes the first term and the sum is the grid effect, which differentiating the control law would add.

The step size $h(\theta)$ is piecewise smooth with jumps: within an acceptance region it varies smoothly through the error estimate, and at its boundary an accepted step is rejected and the entire later grid shifts. Halving the spacing of a sweep in θ doubles the measured magnitude of the grid effect, which identifies it as a jump height divided by $\Delta\theta$, not a derivative. Automatic differentiation reproduces only the smooth part, so differentiating the control law returns the exact gradient plus the smooth part of the grid effect, without the jump part, which is the larger of the two. The result is neither the derivative of the exact solution, to which the addition does not belong, nor that of the reported value, whose dominant contribution is missing. Recovering the grid effect requires a smoothed controller, a fixed grid or a randomised estimator, none of which differentiating the existing chain provides.

Each mode returns the state it integrated together with the derivative of the scheme that produced it, so both follow internal numerical differentiation for the solve they answer. The grids differ from that of a value run in the forward mode: with the default `sensErrCon = TRUE` it maximises its error norm over the sensitivity columns as in (15), integrates on a grid finer than a value run takes, and its values differ from a value run's by $O(\mathrm{tol})$. The default keeps the sensitivities in the error norm because they are integrated alongside the state and their errors accumulate; CVODES guards against this in the same way (`CVodeSetSensErrCon`). The value pass of the reverse mode takes the grid of a value run, as the forward run of CVODES' adjoint sensitivity analysis does, and the reverse gradient is the derivative of the trajectory a value run returns.

Why the state error is not enough

A grid selected from the state error alone need not resolve the linearised flow. The derivative of the step map is accurate where the step resolves the flow on the modes present in the tangents, and the state error measures that only on the modes present in the state, weighted by its own magnitude. On a trajectory at rest it measures nothing: a state that starts at zero and is not driven, or a steady state, is a polynomial the corrector reproduces, its error estimate is zero, and the step grows by the controller's maximum factor on every step. The adjoint on that grid is the exact derivative of a scheme far from the flow. For $\dot{x} = -kx$, $\dot{y} = kx - y$ from $x = y = 0$ over $0, 1, \ldots, 50$, a grid of four steps returns $+0.80$ for a contraction whose value is -0.43. Internal numerical differentiation meets the same failure for $\dot{x} = px$ from $x = 0$ [15, Sec. 6.7.5], and forward automatic differentiation through an adaptive solver meets it unless the controller's norm includes the derivative parts [18]. Where a state is small against `abstol`/`reltol` the effect is weaker but of the same kind. A parameter can also reach the flow where the state does not see it: in $\dot{x} = -kx + \sin(100t)\,q$ at $q = 0$ the state is smooth while $\partial x/\partial q$ follows the fast forcing, and a parameter that enters only through an event's

value acts on the flow from that event on. Such a gradient is resolved by checking the sweep (Section 3.9), not the value pass.

Checking the sweep

`adjointControl(refine = TRUE)` holds the sweep to the tolerances, step by step, on the trajectory the value pass stored. It applies the local error test of the backward problem of CVODES [19], so that a tolerance means the same for both. Step n hands $\lambda(t_n)$ to the step below and adds a share $\Delta\bar{\theta}_n$ to the parameter gradient, whose sum from the end of the run down to t_n is the running gradient $\bar{\theta}(t_n)$. With δ the error estimate of the step's adjoint, the step passes when

$$\begin{gathered}\max\Big(\big\|\delta\lambda(t_n)\big\|_{\mathrm{rms},\,w_\lambda},\ \big\|\delta\Delta\bar{\theta}_n\big\|_{\mathrm{rms},\,w_\theta}\Big) \le 1,\\ w_{\lambda,i} = \texttt{reltol}\,|\lambda_i(t_n)| + \texttt{abstol}, \qquad w_{\theta,i} = \texttt{reltol}\,|\bar{\theta}_i(t_n)| + \texttt{gradtol},\end{gathered} \tag{44}$$

in the weighted root-mean-square norm of (10). The gradient term enters only with `gradtol`, as CVODES holds its quadrature to the error test only with an absolute tolerance on it (`CVodeSetQuadErrConB`). The test is local: the errors of the steps add up over the sweep, as the forward mode's do over the run.

The estimate. Under `rb4` the step's output is $X_6 + e$ and its embedded solution X_6, so the adjoints of the two differ by the response to a seed on e alone. That response is one more backward pass over the same stages and the same factorisation, and estimates the error of the embedded adjoint. A step that meets (44) with it is taken as it is. Otherwise the step is taken again in $m = 2, 4, \dots, 1024$ substeps from its own start state, forward and back with the observations that fall into each, and successive refinements are compared. The difference d_m is divided by Richardson's factor $\min(2^q - 1,\ \max(1,\ d_{m/2}/d_m - 1))$ only as far as two successive differences show the order q; the first halving is taken at face value. `tsit5` has no embedded adjoint and starts with the halving. With `adjointControl(trace = TRUE)`, `$adjoint$substeps` reports the m each step took.

Multistep methods. The share of a `bdf` or `adams` step cannot be recomputed in halves: the Nordsieck array couples the step to the history before it at order one, and replacing one step by substeps changes what every later step reads. The checked sweep integrates the flow's adjoint between the grid points instead, each interval from the stored start state, and hands only the state cotangent across a grid point. Its gradient is the derivative of the flow along the computed trajectory, not of the scheme, and differs from the scheme's derivative by the order of the tolerance, as the continuous adjoint of CVODES does [19].

The flow over an interval of length H is extrapolated linearly implicit Euler, the scheme of SEULEX [9, Sec. IV.9]. Chain j takes $n_j = j$ steps of $h_j = H/n_j$,

$$(h_j^{-1} I - J_0)\, D_k = f(x_k, t_k), \qquad x_{k+1} = x_k + D_k, \tag{45}$$

on the one Jacobian J_0 at the interval's start. A chain is a W-method: it is consistent for any matrix in place of the Jacobian, so its error has an expansion in powers of h_j, and the Aitken-Neville tableau $T_{j,k} = T_{j,k-1} + (T_{j,k-1} - T_{j-1,k-1})/(n_j/n_{j-k} - 1)$ raises the order by one per column. Chain j costs one factorisation of $h_j^{-1} I - J_0$, a change of the diagonal of one Jacobian, n_j right-hand sides and n_j solves.

The extrapolated state $T_{j,j}$ is a fixed linear combination of the chains' end states, so its adjoint is the same combination of the chains' adjoints, and the tableau is built on those: the cotangent of the start state and the interval's share of the gradient, each chain swept back on its own factorisation. A step of (45) hands $\mu_k = (h_j^{-1} I - J_0)^{-\mathsf{T}} \lambda_{k+1}$ to x_k through $J(x_k)^{\mathsf{T}} \mu_k$ and to θ through $f_\theta(x_k)^{\mathsf{T}} \mu_k$, and through J_0 to the interval's start: $\sum_k \mu_k^{\mathsf{T}} (\partial J_0)\, D_k$ is one bilinear form over the Jacobian's pattern, $M_{rc} = \sum_k \mu_{k,r} D_{k,c}$, which the generated code pulls back in one evaluation. The difference e_j of two neighbouring entries $T_{j,j}$, $T_{j,j-1}$ estimates the error of $T_{j,j-1}$; $T_{j,j}$ is returned, and its error is estimated as $e_j \min(1, e_j/e_{j-1})$, the rate of the last two differences applied one column on, as CVODES estimates the error of the solution it returns. The first column whose estimate passes (44) is taken, and an interval the eighth does not pass is halved, its midpoint reached forward under the state's own test. An observation inside the interval cuts it there, the piece after it starting from the state the state's own test reaches.

Limits. A one-step method's step that misses the test at 1024 substeps is taken with 1024, and the halving stops early where from $m = 32$ on a halving no longer shrinks the difference; a multistep interval is halved at most ten times. An error source that the refinement does not reduce, a forcing far faster than the steps, leaves the estimate below the error.

Relation to the forward mode

Forward and reverse derivatives of the same model do not agree to machine precision: the two modes differentiate different discretisations, each exactly, or under `refine` the reverse mode the flow, and the difference falls with the tolerance. The value and the reverse gradient returned to the caller are consistent, which a trust-region method, comparing a predicted with an achieved reduction, relies on.

Of the four methods, the Rosenbrock adjoint is the most accurate: its stages are direct solves, so its adjoint is the exact transpose of the discrete step, whereas the adjoint of a corrector applies the implicit function theorem to an iteration stopped at a finite tolerance.

The reverse mode supports all four methods, time- and root-triggered events, forcings, the batch entry point and the sparse linear solver. It is implemented in `cppde_adjoint_step.hpp`, `cppde_reverse_step.hpp` and `cppde_reverse_trajectory.hpp`.

Second derivatives: forward over reverse

For a fixed cotangent, (22) defines the gradient map

$$\bar{\theta} : \mathbb{R}^M \to \mathbb{R}^M, \qquad \theta \mapsto \nabla_\theta J(\theta), \tag{46}$$

whose Jacobian is the Hessian $\nabla^2_\theta J$. The sweep that evaluates (46) is generic in the scalar type, so instantiating it on `dual`$\langle$`double`$, N\rangle$ instead of `double` differentiates the sweep itself. Seeded with a tangent $V \in \mathbb{R}^{M\times N}$ it returns $(\nabla_\theta J,\ \nabla^2_\theta J\, V)$, both exact rather than difference quotients [20]; for $V = I_M$ the second member is the Hessian. For an ODE this is the second-order adjoint [21], [22]. It is selected by `derivMode = "forward-reverse"` and reported as `curvature`, of shape $[n_x + n_p, N, n_{\text{seed}}]$, beside the `cotangent` and `tangent` of the same run.

The tangents satisfy the second-order adjoint system. For a direction $v = (v_0, v_p)$ in θ, let σ be the directional sensitivity and μ the tangent of λ along v. Then σ solves (12) contracted with v,

$$\dot{\sigma} = \frac{\partial f}{\partial x}\,\sigma + \frac{\partial f}{\partial p}\,v_p, \qquad \sigma(t_0) = v_0, \tag{47}$$

and differentiating (23) along v gives μ backwards from $\mu(T^+) = 0$,

$$\dot{\mu}_i = -\sum_k \frac{\partial f_k}{\partial x_i}\,\mu_k - \sum_k \lambda_k \left(\sum_j \frac{\partial^2 f_k}{\partial x_i\,\partial x_j}\,\sigma_j + \sum_m \frac{\partial^2 f_k}{\partial x_i\,\partial p_m}\,v_m \right), \tag{48}$$

with jumps $\mu(t_o^-) = \mu(t_o^+) + \dot{w}_o$, where $\dot{w}_o$ is the derivative of the cotangent row w_o along v. It is zero for a cotangent independent of θ; a cotangent formed from the solution, such as a weighted residual, supplies it through the `curvature` argument, an array $[n_{\text{out}}, n_x, n_{\text{seed}}, N]$. Without it the gradient is unchanged and the Hessian misses the dependence of the cotangent on θ. Differentiating (26) along v gives the two halves of $\nabla^2_\theta J\, v$: the state half $\mu(t_0)$ and the parameter half

$$\int_{t_0}^{T} \left(\sum_k \frac{\partial f_k}{\partial p_m}\,\mu_k + \sum_k \lambda_k \left(\sum_j \frac{\partial^2 f_k}{\partial p_m\,\partial x_j}\,\sigma_j + \sum_l \frac{\partial^2 f_k}{\partial p_m\,\partial p_l}\,v_l \right) \right) \mathrm{d}t. \tag{49}$$

cppDE does not integrate (47) to (49) but obtains their discrete counterpart from the dual instantiation: the tangent propagated alongside λ is μ on the grid the forward run chose, and the tangent of the parameter accumulator is (49). The second derivatives of f in (48) and (49) come from instantiating the first-order contractions $J^{\mathsf{T}}\lambda$ and $(\partial f/\partial p)^{\mathsf{T}}\lambda$ over duals, so forward over reverse needs no generated expression beyond

those of the first-order sweep. For a Rosenbrock method these already include the contractions of Jv, which are second derivatives of f.

The sweep reads tangents from its checkpoints, so the forward pass is a sensitivity run in the same N directions. Its error control reads the values only, so its grid does not depend on the directions. The cost, one sensitivity integration and one sweep, both of width N, is linear in N. Forward over forward, selected by `derivMode = "forward-forward"` or `deriv2 = TRUE`, propagates the full second-order sensitivity tensor through the integration at a cost quadratic in M. A difference quotient of the gradient would cost N further gradient evaluations and lose half of the available digits.

Forward over forward returns a different object: `hessian`, the second derivatives of every state with respect to every pair of parameters, from which the Hessian of any functional can be contracted afterwards. Forward over reverse returns the Hessian of one functional. For a likelihood the mixed route is cheaper by a factor of M; where the states themselves are of interest, `hessian` is the only route.

The second order needs two further facilities. The iteration matrix has tangents, so the transposed solve applies the implicit function theorem with the matrix transposed alongside; `cppde_ad_lu.hpp` and `cppde_sparse_ad_lu.hpp` provide this in the dense and the sparse case. And a generated model body opens an arena scope of its own, which would release a tangent allocated there for one of the caller's numbers, so the sweep binds the tangent storage of every buffer before passing it to the model, as the stepper does for its own buffers through the tangent slab.

The two second-order routes agree to 10^{-6} on `bdf`, `rb4` and `tsit5`, and all four methods return the same Hessian in reverse to 10^{-5}. The second comparison is the stronger: the four methods adapt grids of their own, so agreement means that each sweep differentiated the discretisation its own method applied.

3.10 Batches of conditions

`solveODEBatch()` solves K conditions of one model in a single `.Call`, in three phases. The main thread reads the arguments of every condition from R into plain C++ structures. OpenMP threads then run the K solves with dynamic scheduling, so a slow condition does not hold up the others; the workers touch neither the R API nor R's allocator, which is not thread-safe. The main thread finally builds the result list. When `times` fixes the output grid, the result arrays are allocated before the parallel phase and the workers write into them directly; a root event leaves the row count open until the solve ends, and its output is copied afterwards.

Each condition is solved as `solveODE()` would solve it: the workers share no state and form no reductions across conditions, so the result is bitwise identical to the serial loop, whatever the thread count and schedule.

The thread count is the smaller of the request and K. It falls to one inside an enclosing OpenMP region, where the caller already runs in parallel, and in a child created by `fork()`, because the OpenMP thread pool does not survive a fork; a `pthread_atfork` handler marks the child. The BLAS library's own threads are set to one for the duration of a batch and restored afterwards. A second `pthread_atfork` handler, installed once per process, sets the BLAS and OpenMP thread counts to one across every `fork()` and restores them in the parent. `forkGuard()` reports which BLAS library was found, and `batchAvailable()` whether a model runs its conditions in parallel.

`prepareBatch()` performs the first phase once and keeps the converted arguments in a handle; `solveBatch()` then solves the same conditions with new parameter values, tangents, Hessians, cotangents, curvatures, adjoint controls or stores of unchanged shapes, as an optimiser does at every iteration; new tolerances need a new `prepareBatch()`. Reverse-mode solves use the same entry with one cotangent per condition, and `cvode()` models provide the same batch entry.

4 `cvode()`

`cvode()` compiles a model against SUNDIALS CVODE/CVODES (≥ 6.0), installed on the build host, and exposes it through the `solveODE()` interface of `cppODE()`.

4.1 What is shared

The code generator, the `events`, `forcings`, `rootfunc` and `fixed` arguments, the returned object and the column order of `parms` and `tangent` are those of `cppODE()`. The Jacobian $J = \partial f/\partial x$ is formed on the expression graph of Section 2 and emitted as for `cppODE()`, linear tables and loops included. Both backends select the sparse KLU solver for $n_x \geq 8$ states and a structural density of J of at most 0.4, provided SuiteSparse was found at install time, and dense LAPACK otherwise; `sparse = TRUE` requires KLU. Forcing functions are PCHIP interpolants, events reset the state discontinuously, and root events fire when a user-supplied scalar function crosses zero. The chain rule $S_\theta = S_p\,\partial p/\partial\theta$ of (13) is that of `cppODE()`, with the same `tangent` shapes (state-only, full, partial) and row-name conventions.

4.2 What differs

Sensitivity right-hand side. CVODES integrates the forward-sensitivity equation

$$\dot{S}_k = J(x,p)\,S_k + \frac{\partial f}{\partial p_k}, \qquad k = 1, \dots, n_p, \tag{50}$$

directly. Its generated right-hand side is one Jacobian-vector product of f in the direction $(S_k, \partial p/\partial\theta_k)$, evaluated in `double` without forming J. The in-tree backend obtains the same equation, (12), by evaluating f on dual-typed state. The generated product avoids the arena allocation of each dual evaluation but runs once per direction; the dual evaluation covers all directions in one pass.

Methods. CVODE provides only the multistep families, selected by `method = "bdf"` or `"adams"`; `rb4` and `tsit5` are unavailable. The stability properties of Section 3 apply unchanged.

Second-order sensitivities. CVODES provides only first-order forward sensitivities, and `cvode()` does not accept `deriv2 = TRUE`.

Events. Events use the Heun transport of `cppODE()` with the first-order corrections (17) and (19). The transport runs in the generated glue, outside CVODES, which re-initialises its sensitivities from the transported state.

4.3 Adjoint sensitivity analysis

`cvode(..., derivMode = "reverse")` compiles the CVODES adjoint sensitivity analysis (ASA) in place of forward sensitivities and returns the `$cotangent` of a native reverse solve: the contraction (22), states first and parameters second. The rest of the call, batch entry point included, is unchanged.

Backward problem. Where `cppODE()` transposes the forward steps, CVODES integrates (23) as an ODE of its own [13]. The generated glue emits both right-hand sides as vector-Jacobian products of f (Section 2), the contractions of the native reverse mode, without forming J:

$$\dot{\lambda} = -\left(\frac{\partial f}{\partial x}\right)^{\mathsf{T}}\lambda, \qquad \dot{q} = -\left(\frac{\partial f}{\partial p}\right)^{\mathsf{T}}\lambda, \tag{51}$$

the second being the quadrature (26) with $q(T) = 0$. CVODES integrates it alongside the backward state and, by default, leaves it out of the error test: with gradient entries of very different scale the test would stall the backward step against `reltol` alone. `adjointControl(gradtol = g)` puts it in, with absolute tolerance g, so the backward step size then holds the gradient's own error. The checked sweep of Section 3.9 applies the same test to the discrete steps. Both equations run from T down to t_0; the minus signs cancel against the reversed limits, so $\lambda(t_0)$ is the state part of the gradient and $q(t_0)$ the parameter part. The backward problem is a BDF solve with the linear solver of the forward problem, KLU for a sparse model, and the analytic Jacobian $-J^\top$, formed from J in the forward pattern, at the caller's tolerances and step budget.

Checkpoints. The backward right-hand side needs $x(t)$, which the forward pass does not keep. CVODES stores a checkpoint every `asaCheckpoints` accepted steps, 200 by default, and recovers the states between two checkpoints by re-integrating forward from the earlier one and interpolating; the argument trades memory

against re-integration. This is the main difference between the two discretisations: the adjoint sees interpolated states, not the ones the forward run stepped through, so the gradient is the derivative of no discretisation actually performed. It approximates the exact gradient to $O(\mathrm{tol})$ and differs by the same order from the derivative of the reported value.

Jumps at observations. λ starts from the cotangent row of the last observation, and the sweep stops at each earlier observation time to add its row, the jump condition of (23). A jump is a new initial condition, so the backward problem is re-initialised where a row is nonzero, and only there: re-initialising at every output time would reduce the method to first order at times the objective does not observe.

Several cotangents. The columns of a cotangent share the checkpoints, but each needs its own backward integration, where the native mode sweeps its stored checkpoints once per column. `keepStore`, `store` and the rest of `adjointControl()` have no counterpart: CVODES holds its own checkpoints and runs its own backward controller, and the continuous adjoint has no discrete steps whose errors could be estimated, weighted or reported.

Events. CVODES cannot be re-initialised in the middle of a forward run it records for the adjoint, so a model with events is integrated in stretches between the jumps, each on its own CVODES memory with its own checkpoints, root functions and linear solver. A jump starts the next stretch from the state after it. The backward sweep runs the stretches last to first, each as its own backward problem started from the adjoint the later stretch handed back, and between two stretches applies the transpose of the jump: for $x^+ = h(x^-, p, t_e)$, $\lambda^- = (\partial_x h)^\mathsf{T}\lambda^+$ and the gradient gains $(\partial_p h)^\mathsf{T}\lambda^+$, together with the terms of an event time that moves with p or, for a root $g(x, p, t) = 0$, with the state, $\delta\tau = -(\nabla_x g\,\delta x + \partial_p g\,\delta p)/\dot{g}$, as in Section 3.9. Root finding is switched off on a stretch before the sweep: CVODES recomputes the forward solution between checkpoints step by step, and a root reported again there would label the stored data with the root's time. A model without events keeps the single stretch.

Restrictions. The same model cannot also provide forward sensitivities, and there is no second-order mode. A Hessian, or a gradient consistent with the value it accompanies, therefore requires the native backend.

As an independent implementation of the same mathematics, ASA checks the in-tree adjoint: the two agree to 10^{-6} on the test models.

5 `cppFUN()`

`cppFUN()` compiles a system of algebraic expressions

$$y_i = g_i(x_1, \dots, x_{n_x}, p_1, \dots, p_{n_p}), \qquad i = 1, \dots, n_y, \tag{52}$$

with its first and second derivatives and no time integration. The variables x are the rows of an $n_{\mathrm{obs}} \times n_x$ matrix, one per observation; the parameters p are shared by all rows. Typical uses are observation maps, which connect integrator output to data and whose derivatives enter the gradient of a likelihood, and reparametrisations $p = \Phi(\theta)$, whose Jacobian is the tangent of (13).

The expressions are parsed into the expression graph of Section 2, from which the generator emits one function template over the scalar type that evaluates g for one observation, instantiated on `double`, on `cppde::dual<double, 0>` and on the nested dual `cppde::dual2nd<double, 0>`. The vector-Jacobian product is a reverse sweep on the same graph, emitted as a second template and instantiated on `double` and on the dual.

Every entry runs compiled code. With the default `compile = FALSE` the constructor only writes the source; the object must be passed to `compile()` before use, and its functions stop with an error until then.

5.1 Entry points and derivative directions

`derivMode` names the derivative products built, any of `"forward"`, the default, `"reverse"` and `"forward-reverse"`, which implies `"reverse"`. A product left out costs no entries and no compile time. `deriv` and

`deriv2` switch the first- and second-order entries; `deriv2 = TRUE` requires `"forward"`. Table 4 lists the R functions of the returned object and the compiled entries they call.

R function	direction	compiled entry
`func(x, p)`	any	`_eval_c`
`jac(x, p[, tangentX, tangentP])`	forward	`_eval_ad_c`
`hess(x, p[, tangentX, tangentP, hessianX, hessianP])`	forward	`_eval_ad2_c`
`evaluate(…, deriv2)`	forward	`_eval_ad_c` or `_eval_ad2_c`
`evaluateBatch(sets, cores, deriv2)`	forward	`_eval_ad_batch` or `_eval_ad2_batch`
`vjp(x, p, cotangent)`	reverse	`_vjp_c`
`vjp(x, p, cotangent, tangentX, tangentP, curvature)`	forward-reverse	`_vjp_ad_c`

Table 4: R functions of a `cppFUN()` object and the compiled entries they call, each prefixed by the model name; `x` and `p` stand for the arguments `vars` and `params`. The `_c` entries are `.Call` interfaces that take their arguments by reference; `_eval`, `_eval_ad`, `_eval_ad2` and `_vjp` also exist as `.C` interfaces, used only for a shared object without the `.Call` symbol.

Forward direction. `jac` evaluates g on duals and returns $\partial y/\partial\theta$ as an array $[n_{\text{obs}}, n_y, n_\theta]$; `hess` evaluates it on nested duals and returns $\partial^2 y/\partial\theta^2$ as $[n_{\text{obs}}, n_y, n_\theta, n_\theta]$. The directions θ are set by the tangents of the next section. Without them the identity over the variables and the differentiated parameters is used, which gives the Jacobian or Hessian with respect to (x, p). `evaluate` returns value and derivatives from one pass as `y`, `tangent` and, with `deriv2 = TRUE`, `hessian`; separate calls of `func`, `jac` and `hess` would evaluate g up to three times. `evaluateBatch` runs `evaluate` over a list of argument sets in one call, distributed over OpenMP threads with dynamic scheduling, and falls back to a loop for fewer than two sets.

Reverse direction. For a cotangent w of the outputs, $[n_{\text{obs}}, n_y]$ or $[n_{\text{obs}}, n_y, n_{\text{seed}}]$, `vjp` returns

$$(w_x)_{ojs} = \sum_i w_{ois} \frac{\partial g_i}{\partial x_j}(x_o, p), \qquad (w_p)_{ks} = \sum_o \sum_i w_{ois} \frac{\partial g_i}{\partial p_k}(x_o, p), \tag{53}$$

as `cotangentX` and `cotangentP`, together with the value y; the parameter part sums over the observations, which share p. The contraction is one reverse sweep per observation and cotangent column, at a cost independent of the number of upstream parameters, and forms neither the Jacobian nor the Hessian. Statements of the sweep that do not depend on the cotangent are evaluated once per observation, outside the loop over cotangent columns.

Given the tangents v_x and v_p of the inputs in n_{dir} directions as `tangentX` and `tangentP`, or the derivative $\dot{w}$ of the cotangent along them as `curvature`, `vjp` runs the same template on duals and additionally returns

$$\frac{\mathrm{d}}{\mathrm{d}v}(w^\mathsf{T} J) = w^\mathsf{T} \frac{\partial J}{\partial(x,p)}\, v + \dot{w}^\mathsf{T} J, \qquad J = \frac{\partial g}{\partial(x,p)}, \tag{54}$$

as `curvatureX` and `curvatureP`. This is forward over reverse (Section 3.9): an observation map contributes its curvature to a backward chain without forming a Hessian.

5.2 Tangent, hessian, cotangent and curvature

A map has a tangent and a Hessian, a functional a cotangent and a curvature. For the upstream map $\Phi : \theta \mapsto (x, p)$ these are the `tangent` $\partial\Phi(\theta)$ and the `hessian` $\partial^2\Phi(\theta)$; for a scalar functional L of the outputs y, the `cotangent` $\partial L(y)$ and the `curvature` $\partial^2 L(y)\,\partial y$, the derivative of the cotangent along the tangent. An argument and its result share a name (Table 5): a forward argument lives on the inputs and its result on the outputs, a reverse argument on the outputs and its result on the inputs.

Forward over forward takes the `hessian` of the inputs and returns that of the outputs, the second derivatives of every output in every pair of tangent directions. Forward over reverse takes the `curvature` of the outputs and returns that of the inputs, the Hessian of the seeded functional applied to the tangent, or, with the identity tangent, the columns of that Hessian.

`solveODE()`	`cppFUN()` argument	`cppFUN()` result
`tangent`	`tangentX`, `tangentP`	`tangent`
`hessian`	`hessianX`, `hessianP`	`hessian`
`cotangent`	`cotangent`	`cotangentX`, `cotangentP`
`curvature`	`curvature`	`curvatureX`, `curvatureP`

Table 5: Names of the four derivative objects in R. `solveODE()` uses one name for the argument and the element of its result; the `cppFUN()` results are elements of the lists that `evaluate` and `vjp` return.

Chain rule. `cppFUN()` takes the tangent and the hessian of its inputs in two blocks each:

- `tangentX[obs, name, theta]`: $\partial x_i/\partial\theta_a$, per observation row, variable and direction θ_a;
- `tangentP[name, theta]`: $\partial p_j/\partial\theta_a$, the same for all observations;
- `hessianX[obs, name, theta1, theta2]`: $\partial^2 x_i/\partial\theta_a\partial\theta_b$, optional, zero by default;
- `hessianP[name, theta1, theta2]`: $\partial^2 p_j/\partial\theta_a\partial\theta_b$, optional, zero by default.

They may also be supplied as the attributes `deriv` and `deriv2` of the variables and parameters. Tangent rows of parameters in `fixed` are zero, and symbols absent from the tangent contribute zero.

With the tangent $\partial\Phi \in \mathbb{R}^{n_{\text{obs}}\times n_{\text{diff}}\times n_\theta}$ over the $n_{\text{diff}} = n_x + n_p$ inputs and the hessian $\partial^2\Phi$, the chain rule gives the Jacobian

$$\left.\frac{\partial y}{\partial\theta}\right|_{\text{obs}} = J(x_{\text{obs}}, p)\cdot\partial\Phi_{\text{obs}}, \tag{55}$$

and the Hessian of output o

$$\left.\frac{\partial^2 y_o}{\partial\theta\,\partial\theta^\top}\right|_{\text{obs}} = \partial\Phi_{\text{obs}}^\top H_o(x_{\text{obs}}, p)\,\partial\Phi_{\text{obs}} + \sum_i J_{o,i}(x_{\text{obs}}, p)\,\partial^2\Phi_{\text{obs},i,\cdot,\cdot}, \tag{56}$$

whose second term vanishes when Φ is affine.

Neither J nor H is formed: both contractions follow from seeding the tangent slots of the duals. Each input scalar x_i becomes

$$\tilde{x}_i = (x_i + \partial\Phi_{i,\cdot}\,\varepsilon_{\text{in}}) + (\partial\Phi_{i,\cdot} + \partial^2\Phi_{i,\cdot,\cdot}\,\varepsilon_{\text{in}})\,\varepsilon_{\text{out}},$$

a nested dual whose inner tangent slots hold $\partial x_i/\partial\theta$ and whose outer tangent slot k is an inner dual with value $\partial x_i/\partial\theta_k$ and inner tangents $\partial^2 x_i/\partial\theta_k\partial\theta$. The multiplication rule then propagates the $\partial\Phi^\top H\,\partial\Phi$ and $J\cdot\partial^2\Phi$ terms in one evaluation; the inner tangents of $\tilde{y}$ are $\partial y/\partial\theta$, and the cross slots $\tilde{y}[k][m]$ are $\partial^2 y/\partial\theta_k\partial\theta_m$.

5.3 Pass-through of unmodelled inputs

With `attach.input = TRUE`, inputs that do not appear among the variables and parameters of (52) are appended to the output unchanged: their values are copied, their Jacobian rows are zero for a variable and the identity for a parameter, and their Hessian rows are zero. An observation map then need not repeat every column of the integrator output as an identity, as with the partial-shape `tangent` of `solveODE()`.

6 Benchmarks

Both backends take the right-hand side, Jacobian, events and linear-solver choice from the same expression graph and run at the same tolerances, so a head-to-head measurement compares the numerical cores.

Setup. The suite comprises 43 problems (Table 6): 33 published systems-biology models of the PEtab benchmark collection [23], [24] and 10 classical test problems, with $n_x = 2$ to 504 states. First-order sensitivities run forward for up to $M = 32$ parameters and in reverse on the two models with $M \geq 120$, Lang and Chen. Both backends use BDF/NDF with the automatically selected linear solver at $(\mathrm{rtol}, \mathrm{atol}) \in \{(10^{-4}, 10^{-6}), (10^{-7}, 10^{-9}), (10^{-10}, 10^{-12})\}$, and a cell reports the median of five batches of at least 0.25 s. The suite ran on AMD EPYC 7513 nodes with twelve workers each; both backends of a cell ran in the same worker. Ratios are geometric means over the three tolerances; Figure 2 shows $t_{\texttt{cvode()}}/t_{\texttt{cppODE()}}$ per problem.

problem	n_x	M	problem	n_x	M	problem	n_x	M
van der Pol†	2	1	Rahman 2016	7	9	Bachmann 2011	25	27
Bertozzi 2020	3	3	Boehm 2014	8	6	Isensee 2018	25	32
Borghans 1997	3	20	Elowitz 2000	8	18	Lucarelli 2018	33	32
OREGO†	3	3	HIRES†	8	2	Laske 2019	34	6
Robertson†	3	3	Okuonghae 2020	8	14	Alkan 2018	36	32
Armistead 2024	4	10	Weber 2015	8	26	Brusselator, N=24†	48	3
E5†	4	3	Brännmark 2010	9	14	FitzHugh-Nagumo, N=24†	48	5
Perelson 1996	4	2	Fujita 2010	9	16	Giordano 2020	51	32
Zhao 2020	4	21	Oliveira 2021	10	10	Salazar-Cavazos 2020	75	6
Beer 2014	5	6	Schwen 2014	11	13	Lang 2024	124	164
Crauste 2017	5	12	Raia 2011	14	18	Brusselator, N=64†	128	3
Fiedler 2016	6	12	Zheng 2012	15	32	FitzHugh-Nagumo, N=64†	128	5
Sneyd 2002	6	14	Blasi 2016	16	8	Chen 2009	504	152
Bruno 2016	7	7	Pollution†	20	25			
Liu 2025	7	7	Raimúndez 2020	22	32			

Table 6: Benchmark problems with their numbers of states n_x and differentiated parameters M. PEtab models are named by first author and year; † marks the classical test problems, van der Pol at $\mu = 1000$ and the one-dimensional Brusselator and FitzHugh-Nagumo systems on N grid points.

Plain solves. `cppODE()` is faster on 41 of 43 problems, by a geometric mean of 1.15, with about as many steps as CVODE (ratios of accepted steps and right-hand-side evaluations 1.02 and 1.06). The margin is a cost per step: the model functions are inlined into the stepper, whereas CVODE calls them through function pointers and dispatches every vector operation through the `N_Vector` table.

Forward sensitivities. `cppODE()` is faster on 34 of 41 problems, by 1.29, and the ratio grows with M: 1.01 for $M \leq 3$, 1.22 for $4 \leq M \leq 12$ and 1.52 for $13 \leq M \leq 32$. CVODES evaluates one Jacobian-vector product per parameter; the in-tree backend evaluates f once on a dual state that holds all M tangents and needs about one eleventh of the right-hand-side evaluations. The dual evaluation has a cost per state component independent of M, so problems with few parameters fall below parity: six of the seven slower ones have $M \leq 6$, the lowest being Salazar-Cavazos ($n_x = 75$, $M = 6$) at 0.67.

Reverse sensitivities. Measured in plain solves, the cost of one gradient by forward sensitivities is linear in M (Figure 3): $1.8 + 0.60\,M$ for `cppODE()` and $1.2 + 0.84\,M$ for CVODES, about 92 and 129 at $M = 152$. The discrete adjoint of Section 3.9 costs 6.3 plain solves on Chen and 3.6 on Lang, independent of M. The CVODES adjoint of Section 4.3, measured on Lang, costs 27 plain CVODE solves and takes 8.8 to 13.7 times as long as the discrete adjoint.

A separate run on the six models with $M \geq 40$, one core per model, compares the two adjoints directly (Figure 4). The CVODES adjoint takes 8.7 to 18.8 times as long on Lang, Lucarelli and Raim'undez, and 1.0 to 1.5 times on Zheng ($n_x = 15$), where a solve takes about a millisecond. The discrete adjoint runs backward over the steps of the forward solve, whereas CVODES chooses its own backward steps and recomputes the

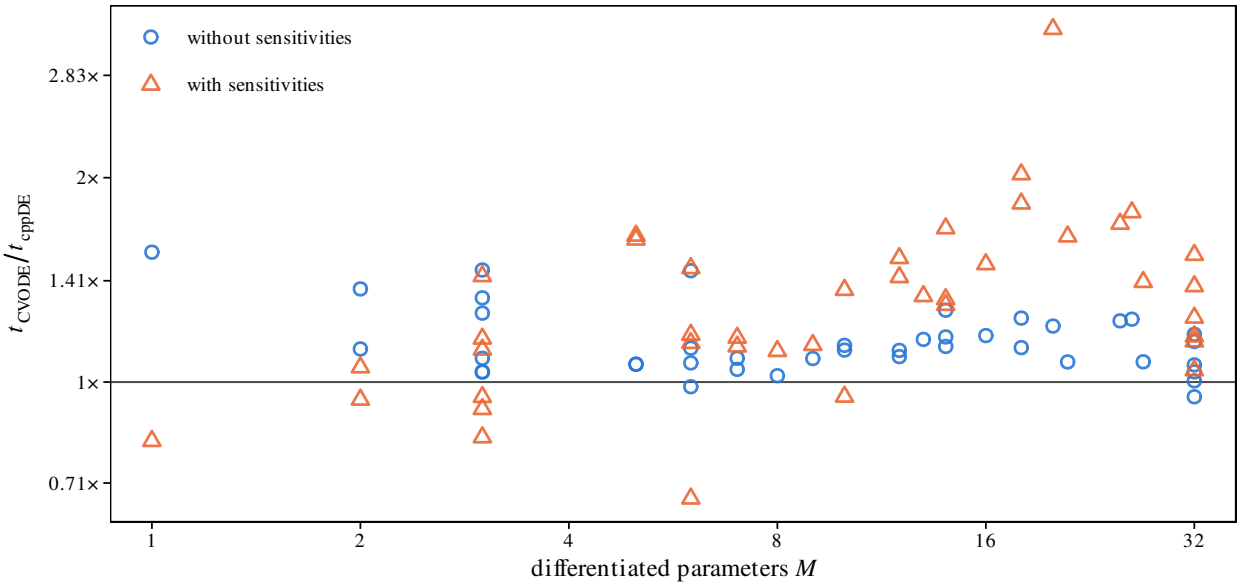


Figure 2: Head-to-head ratio of the two backends, one marker per problem and mode; above the line `cppODE()` is faster. The plain solve is placed at the M of its sensitivity counterpart.

forward state between checkpoints. At rtol $= 10^{-10}$ both gradients deviate from the reference by 10^{-10} to $5 \cdot 10^{-7}$.

Second order. The Hessian of the summed outputs is measured for `cppODE()` alone, on models with at most 30 states: forward over reverse (Section 3.9) with up to 32 directions, forward over forward up to 10 states and 10 directions. Fitted in log-log, the cost grows with exponent 1.9 forward over forward and 0.6 forward over reverse (Figure 5). The two break even at $M = 3$; forward over reverse is 3.0 times faster at $M = 6$ and 9 to 14 times at $M = 9$ and 10. At rtol $= 10^{-10}$ the Hessians deviate from the forward-over-forward Hessian at the reference tolerance by a median of $8.1 \cdot 10^{-10}$ (forward over forward) and $1.8 \cdot 10^{-8}$ (forward over reverse).

Linear solver. Seventeen models were solved again with the linear solver pinned to dense and to sparse. The sparse gain is the same for both backends and grows from 1.09 at $n_x = 8$ to 3.3 at $n_x = 504$ (Figure 6); the automatic selection chose the sparse path in every case.

Accuracy. Against a reference at atol $= 10^{-14}$, rtol $= 10^{-12}$ the error of `cppODE()` is a median 0.99 times that of CVODE without sensitivities (central 90 % of the problems between 0.51 and 1.51) and 1.09 with sensitivities (between 0.66 and 2.37). The reference is itself a CVODE solution, which favours CVODE.

Failures. Six of the 1083 cells failed, all at rtol $= 10^{-10}$: the forward sensitivities of `cppODE()` on Oliveira with each linear solver and on Weber, and forward over forward on the same two models.

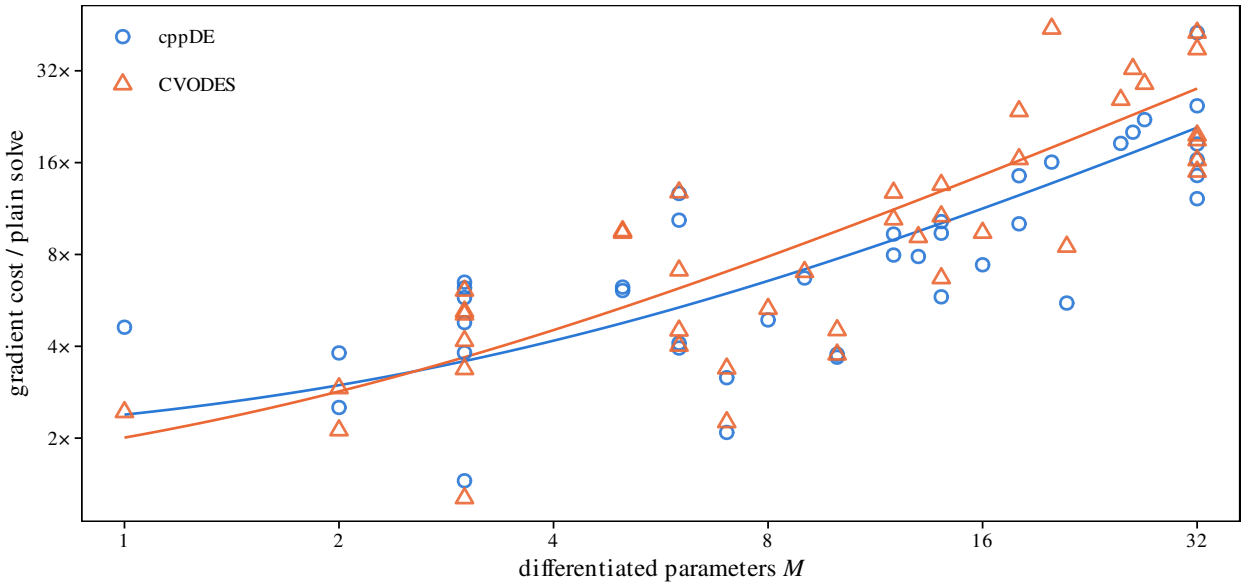


Figure 3: Cost of one gradient by forward sensitivities in units of the plain solve of the same problem and backend, with least-squares lines.

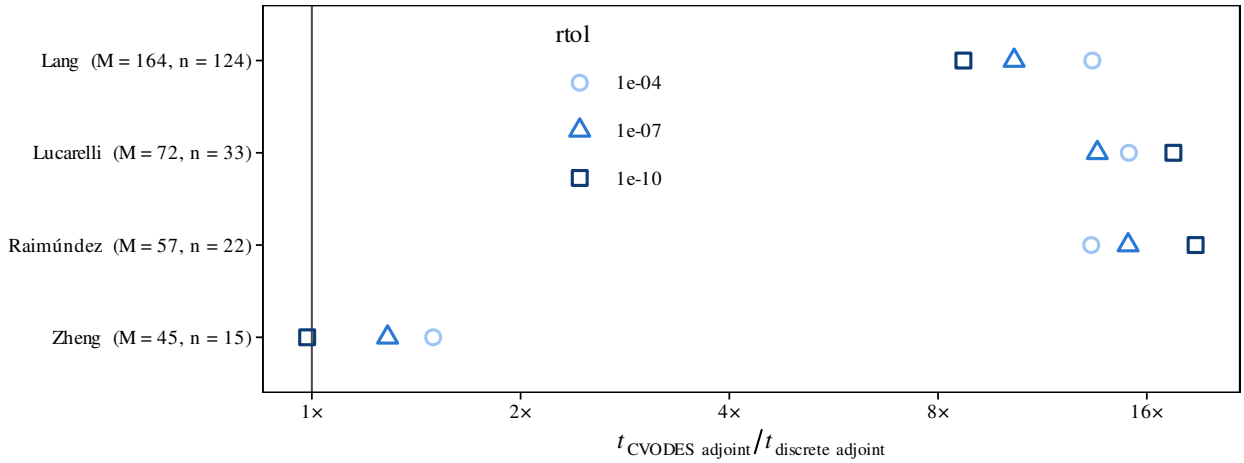


Figure 4: Time of one gradient by the CVODES adjoint over the time by the discrete adjoint, per model and tolerance; right of the line the discrete adjoint is faster.

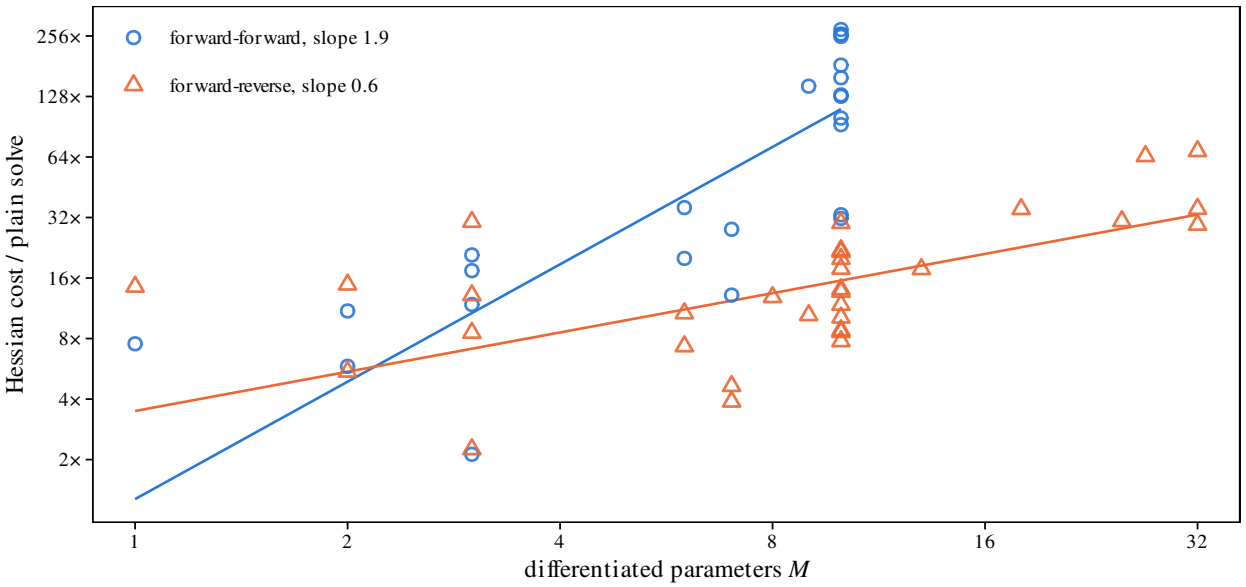


Figure 5: Cost of one Hessian of the summed outputs in units of the plain solve. The lines are power laws fitted in log-log over $M > 1$.

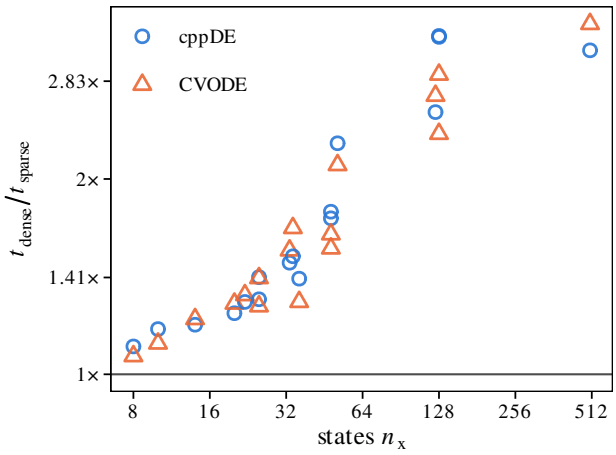


Figure 6: Gain of the sparse over the dense linear solver, one marker per model and backend.

References

[1] A. Griewank and A. Walther, *Evaluating derivatives: Principles and techniques of algorithmic differentiation*, 2nd ed. in Other titles in applied mathematics, no. 105. SIAM, 2008. doi: 10.1137/1.9780898717761.

[2] A. P. Ershov, "On programming of arithmetic operations," *Communications of the ACM*, vol. 1, no. 8, pp. 3–6, 1958, doi: 10.1145/368892.368907.

[3] J.-C. Filliâtre and S. Conchon, "Type-safe modular hash-consing," in *Proceedings of the 2006 workshop on ML*, ACM, 2006, pp. 12–19. doi: 10.1145/1159876.1159880.

[4] A. R. Curtis, M. J. D. Powell, and J. K. Reid, "On the estimation of sparse Jacobian matrices," *IMA Journal of Applied Mathematics*, vol. 13, no. 1, pp. 117–119, 1974, doi: 10.1093/imamat/13.1.117.

[5] T. F. Coleman and J. J. Moré, "Estimation of sparse Jacobian matrices and graph coloring problems," *SIAM Journal on Numerical Analysis*, vol. 20, no. 1, pp. 187–209, 1983, doi: 10.1137/0720013.

[6] A. H. Gebremedhin, F. Manne, and A. Pothen, "What color is your Jacobian? Graph coloring for computing derivatives," *SIAM Review*, vol. 47, no. 4, pp. 629–705, 2005, doi: 10.1137/S0036144504444711.

[7] A. C. Hindmarsh *et al.*, "SUNDIALS: Suite of nonlinear and differential/algebraic equation solvers," *ACM Transactions on Mathematical Software (TOMS)*, vol. 31, no. 3, pp. 363–396, 2005, doi: 10.1145/1089014.1089020.

[8] L. F. Shampine and M. W. Reichelt, "The MATLAB ODE suite," *SIAM Journal on Scientific Computing*, vol. 18, no. 1, pp. 1–22, 1997, doi: 10.1137/S1064827594276424.

[9] E. Hairer and G. Wanner, *Solving ordinary differential equations II: Stiff and differential-algebraic problems*, 2nd ed. in Springer series in computational mathematics, no. 14. Springer, 1996. doi: 10.1007/978-3-642-05221-7.

[10] Ch. Tsitouras, "Runge–Kutta pairs of order 5(4) satisfying only the first column simplifying assumption," *Computers & Mathematics with Applications*, vol. 62, no. 2, pp. 770–775, 2011, doi: 10.1016/j.camwa.2011.06.002.

[11] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving ordinary differential equations I: Nonstiff problems*, 2nd ed. in Springer series in computational mathematics, no. 8. Springer, 1993. doi: 10.1007/978-3-540-78862-1.

[12] U. Naumann, *The art of differentiating computer programs: An introduction to algorithmic differentiation*. in Software, environments, and tools, no. 24. SIAM, 2012. doi: 10.1137/1.9781611972078.

[13] Y. Cao, S. Li, L. Petzold, and R. Serban, "Adjoint sensitivity analysis for differential-algebraic equations: The adjoint DAE system and its numerical solution," *SIAM Journal on Scientific Computing*, vol. 24, no. 3, pp. 1076–1089, 2003, doi: 10.1137/S1064827501380630.

[14] H. G. Bock, "Numerical treatment of inverse problems in chemical reaction kinetics," in *Modelling of chemical reaction systems*, vol. 18, K. H. Ebert, P. Deuflhard, and W. Jäger, Eds., in Springer series in chemical physics, vol. 18., Berlin, Heidelberg: Springer, 1981, pp. 102–125. doi: 10.1007/978-3-642-68220-9_8.

[15] J. Albersmeyer, "Adjoint-based algorithms and numerical methods for sensitivity generation and optimization of large scale dynamic systems," PhD thesis, Ruprecht-Karls-Universität Heidelberg, 2010. Available: https://archiv.ub.uni-heidelberg.de/volltextserver/11651/

[16] P. Eberhard and C. Bischof, "Automatic differentiation of numerical integration algorithms," *Mathematics of Computation*, vol. 68, no. 226, pp. 717–732, 1999, doi: 10.1090/S0025-5718-99-01027-3.

[17] M. Alexe and A. Sandu, "On the discrete adjoints of adaptive time stepping algorithms," *Journal of Computational and Applied Mathematics*, vol. 233, no. 4, pp. 1005–1020, 2009, doi: 10.1016/j.cam.2009.08.109.

[18] F. Sapienza *et al.*, "Differentiable programming for differential equations: A review." [Online]. Available: https://arxiv.org/abs/2406.09699

[19] R. Serban and A. C. Hindmarsh, "CVODES: The sensitivity-enabled ODE solver in SUNDIALS," in *Proceedings of IDETC/CIE 2005, volume 6: 5th international conference on multibody systems, nonlinear dynamics, and control*, Long Beach, CA: ASME, 2005, pp. 257–269. doi: 10.1115/DETC2005-85597.

[20] B. A. Pearlmutter, "Fast exact multiplication by the Hessian," *Neural Computation*, vol. 6, no. 1, pp. 147–160, 1994, doi: 10.1162/neco.1994.6.1.147.

[21] Z. Wang, I. M. Navon, F. X. Le Dimet, and X. Zou, "The second order adjoint analysis: Theory and applications," *Meteorology and Atmospheric Physics*, vol. 50, no. 1–3, pp. 3–20, 1992, doi: 10.1007/BF01025501.

[22] D. B. Özyurt and P. I. Barton, "Cheap second order directional derivatives of stiff ODE embedded functionals," *SIAM Journal on Scientific Computing*, vol. 26, no. 5, pp. 1725–1743, 2005, doi: 10.1137/030601582.

[23] H. Hass, C. Loos, E. Raimúndez-Álvarez, J. Timmer, J. Hasenauer, and C. Kreutz, "Benchmark problems for dynamic modeling of intracellular processes," *Bioinformatics*, vol. 35, no. 17, pp. 3073–3082, 2019, doi: 10.1093/bioinformatics/btz020.

[24] L. Schmiester *et al.*, "PEtab: Interoperable specification of parameter estimation problems in systems biology," *PLOS Computational Biology*, vol. 17, no. 1, p. e1008646, 2021, doi: 10.1371/journal.pcbi.1008646.

A BDF and NDF coefficients

Source. Shampine and Reichelt [8], Table 1; implemented in `cppde_multistepper.hpp`.

The NDF of order q adds to the BDFq corrector a term proportional to the Nordsieck predictor error,

$$\sum_{m=1}^{q} \frac{1}{m} \nabla^m x_n \;-\; h_n\, f(x_n, t_n) \;-\; \kappa_q\, \gamma_q\, (x_n - x_n^{\mathrm{pred}}) \;=\; 0, \qquad \gamma_q \;=\; \sum_{m=1}^{q} \tfrac{1}{m}.$$

Setting $\kappa_q = 0$ recovers BDFq. Table 7 lists the κ_q of cppDE with the step-size gain over BDFq and the stability angle. The argument `useNDF` (default `TRUE`) selects NDF or BDF at compile time.

q	κ_q	Step-size gain over BDFq	Stability angle
1	-0.1850	$+26\,\%$	A-stable
2	-0.1111	$+26\,\%$	A-stable
3	-0.0823	$+26\,\%$	80°
4	-0.0415	$+12\,\%$	66°
5	0.0000	$0\,\%$	51° (= BDF5)

Table 7: NDF correction coefficients κ_q adopted by cppDE, the empirical step-size gain over classical BDFq at matched local error, and the resulting stability angle.

NDF5 is BDF5 ($\kappa_5 = 0$), because the BDF5 stability angle of 51° leaves no room for further reduction.

B Adams-Moulton coefficients

Source. Hindmarsh et al. [7], the ℓ-vector formulation of CVODE; implemented in `cppde_multistepper.hpp` by `adams_set_coefficients()`.

The Adams-Moulton coefficients depend on the step-size history, so no static table holds them: they are recomputed every step from the Nordsieck array. With $\xi_j = (t_n - t_{n-j})/h_n$, the recurrence

$$\ell_0(q) \;=\; 1, \qquad \ell_j(q) \;=\; \ell_{j-1}(q) \;+\; \xi_j^{-1}, \qquad j = 1, \dots, q,$$

followed by a polynomial expansion, yields the corrector weights $\beta_0, \dots, \beta_q$ of the current step. For $h_n = h_{n-1} = \cdots$ they reduce to the fixed-step values of Hairer, Nørsett and Wanner [11], Table III.1.1.

The maximum order is $q_{\max} = 12$, the CVODE default; the order controller selects $q \in \{1, \dots, 12\}$ at run time.

C Rosenbrock4 coefficients

Source. Hairer and Wanner [9, Sec. IV.7], the default coefficients of the `rosenbrock4` stepper of Boost.Numeric.Odeint except the sign of d_4, which is that of `rodas.f`; implemented in `default_rosenbrock_coefficients` in `cppde_rosenbrock4.hpp`.

With iteration matrix $W = (\gamma h_n)^{-1} I - J(x_n, t_n)$, a Rosenbrock method of s stages with diagonal coefficient γ has the stage equations

$$W\, k_i \;=\; f\!\left(x_n + \sum_{j<i} \alpha_{ij}\, k_j,\; t_n + c_i\, h_n \right) \;+\; \frac{1}{h_n} \sum_{j<i} \frac{\gamma_{ij}}{\gamma}\, k_j \;+\; h_n\, d_i\, \frac{\partial f}{\partial t},$$

$$x_{n+1} \;=\; x_n + \sum_{i=1}^{s} m_i\, k_i, \qquad \hat{x}_{n+1} \;=\; x_n + \sum_{i=1}^{s} \hat{m}_i\, k_i.$$

cppDE uses $s = 6$ and $\gamma = 1/4$. The tables are indexed as in the source code: γ, the time partials and the node positions in Table 8, the stage couplings in Tables 9 and 10, the sixth-row weights in Table 11 and the dense-output coefficients in Table 12.

γ	0.25	d_1	0.25	c_2	0.386
d_2	−0.1043	d_3	0.1035	c_3	0.21
d_4	−0.0362			c_4	0.63

Table 8: Rosenbrock4 diagonal coefficient γ, time-partial coefficients d_i, and stage node positions c_i.

	$j = 1$	$j = 2$	$j = 3$	$j = 4$
$\alpha_{2,j}$	1.5440000000			
$\alpha_{3,j}$	0.9466785281	0.2557011699		
$\alpha_{4,j}$	3.3148251871	2.8961240160	0.9986419140	
$\alpha_{5,j}$	1.2212245092	6.0191344813	12.5370833293	−0.6878860361

Table 9: Rosenbrock4 stage-coupling coefficients α_{ij}. Row index i is the stage being formed; column index j ranges over earlier stages.

	$j = 1$	$j = 2$	$j = 3$	$j = 4$
$\gamma_{2,j}$	−5.6688000000			
$\gamma_{3,j}$	−2.4300933568	−0.2063599157		
$\gamma_{4,j}$	−0.1073529058	−9.5945622510	−20.4702861481	
$\gamma_{5,j}$	7.4964433140	−10.2468043146	−33.9999035282	11.7089089321

Table 10: Rosenbrock4 stage-coupling coefficients γ_{ij}, appearing in the $(1/h_n)\sum_{j<i}(\gamma_{ij}/\gamma)\,k_j$ term of the stage equation. Row and column conventions match Table 9.

The step solution and the embedded error estimate are

$$x_{n+1} \;=\; x_n + \sum_{i=1}^{5} \alpha_{6,i}\, k_i, \qquad e_{n+1} \;=\; \sum_{i=1}^{5} \gamma_{6,i}\, k_i,$$

with the sixth-row coefficients of Table 11.

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$
$\alpha_{6,i}$	1.2212245092	6.0191344813	12.5370833293	−0.6878860361	1.0
$\gamma_{6,i}$	8.0832467959	−7.9811329881	−31.5215943287	16.3193054312	−6.0588182388

Table 11: Rosenbrock4 sixth-row coefficients. The $\alpha_{6,i}$ values combine the stage values into the advanced solution x_{n+1}; the $\gamma_{6,i}$ values supply the embedded order-3 error estimate e_{n+1}.

The order-3 interpolant on $[t_n, t_{n+1}]$ combines the five stage values and the step solution through the coefficient sets $d_i^{(2)}$ and $d_i^{(3)}$ of Table 12.

	$i=1$	$i=2$	$i=3$	$i=4$	$i=5$
$d_i^{(2)}$	10.1262350834	−7.4879958776	−34.8009186156	−7.9927717076	1.0251377233
$d_i^{(3)}$	−0.6762803393	6.0877146517	16.4308432089	24.7672251142	−6.5943891257

Table 12: Rosenbrock4 dense-output coefficients $d_i^{(2)}$ and $d_i^{(3)}$. Together with the stage values they form the order-3 interpolant on $[t_n, t_{n+1}]$ used to serve user output points falling between accepted steps.

D Tsitouras 5(4) coefficients

Source. Tsitouras [10], Table 1 and eqs. (7), (8); implemented in `cppde_tsit5.hpp`.

The Tsitouras 5(4) pair (Table 13) has $s = 7$ stages and satisfies the first column simplifying assumption $\sum_i b_i a_{ij} = b_j(1 - c_j)$ for every j, but not the row condition $\sum_j a_{ij} c_j = c_i^2/2$. The seventh stage is the FSAL stage: the row $a_{7,j}$ of the A matrix coincides with b_j for $j = 1, \dots, 6$.

i	c_i	a_{i1}	a_{i2}	a_{i3}	a_{i4}	a_{i5}	a_{i6}
1	0						
2	0.161	0.161					
3	0.327	−0.0084806555	0.3354806555				
4	0.9	2.8971530571	−6.3594484900	4.3622954329			
5	0.9800255409	5.3258648284	−11.7488835641	7.4955393429	−0.0924950664		
6	1.0	5.8614554429	−12.9209693178	8.1593678986	−0.0715849733	−0.0282690504	
7	1.0	0.0964607668	0.01	0.4798896504	1.3790085741	−3.2900695154	2.3247105241

Table 13: Tsitouras 5(4) nodes c_i and coefficients a_{ij}. Row 7 equals the order-5 weights b_j (FSAL).

The seventh stage $k_7 = f(x_{n+1},\, t_n + h_n)$ is reused as k_1 of step $n + 1$. Since $b_7 = 0$ and $\hat{b}_7 \neq 0$, it enters the embedded order-4 estimate but not the order-5 solution. The step controller uses the local-error estimate

$$e_n \;=\; h_n \sum_{i=1}^{7} (\hat{b}_i - b_i)\, k_i .$$

Table 14 lists the weights and their differences.

i	b_i	$\hat{b}_i$	$\hat{b}_i - b_i$
1	0.0964607668	0.0946807557	−0.0017800111
2	0.01	0.0091835655	−0.0008164345
3	0.4798896504	0.4877705284	0.0078808780
4	1.3790085741	1.2342975669	−0.1447110072
5	−3.2900695154	−2.7077123499	0.5823571655
6	2.3247105241	1.8666284182	−0.4580821059
7	0	0.0151515152	0.0151515152

Table 14: Tsitouras 5(4) weights of order 5 (b_i) and 4 ($\hat{b}_i$). The differences are stored in `cppde_tsit5.hpp` as `e1`, ..., `e7` and sum to zero.